

SECR2033
Computer Organization and Architecture

Lecture slides prepared by “Computer Organization and Architecture”, 9/e, by William Stallings, 2013.

Module 2

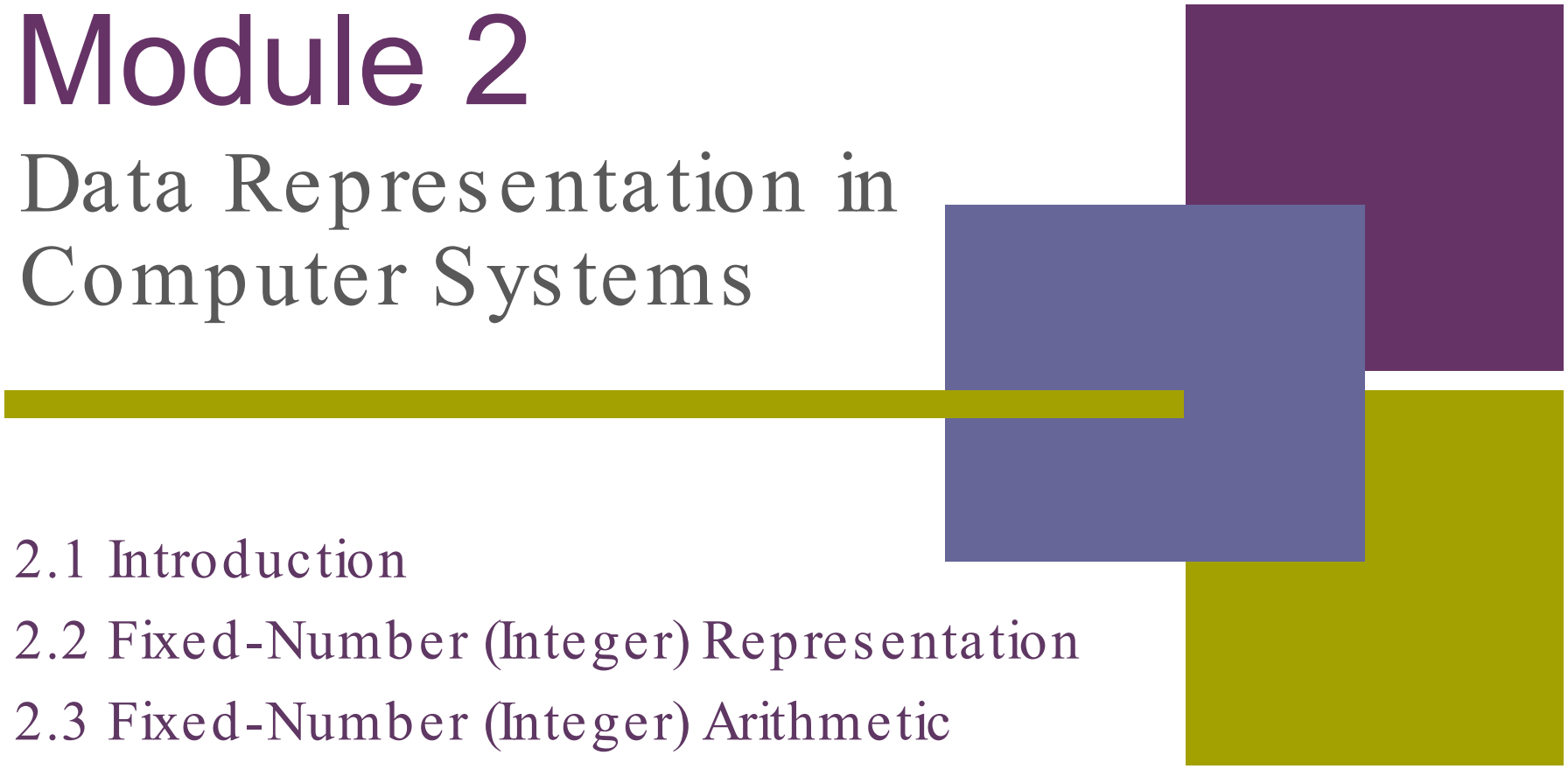
Data Representation in Computer Systems

Objectives:

- To understand the fundamentals of **numerical data** representation and manipulation in digital computers.
- To master the skill of converting between various **radix systems**.
- To understand how **errors** can occur in computations because of overflow and truncation.
- To understand the fundamental concepts of **floating-point** representation.

Module 2

Data Representation in Computer Systems



2.1 Introduction

2.2 Fixed-Number (Integer) Representation

2.3 Fixed-Number (Integer) Arithmetic

2.4 Floating-Points Representation

2.5 Floating-Points Arithmetic

2.6 Summary

Module 2

Data Representation in Computer Systems



2.1 Introduction

- The Arithmetic and Logic Unit

2.2 Fixed-Number (Integer) Re

2.3 Fixed-Number (Integer) Ari

2.4 Floating-Points Representa

2.5 Floating-Points Arithmetic

2.6 Summary

2.1 Introduction

Numbers are represented by binary bits:

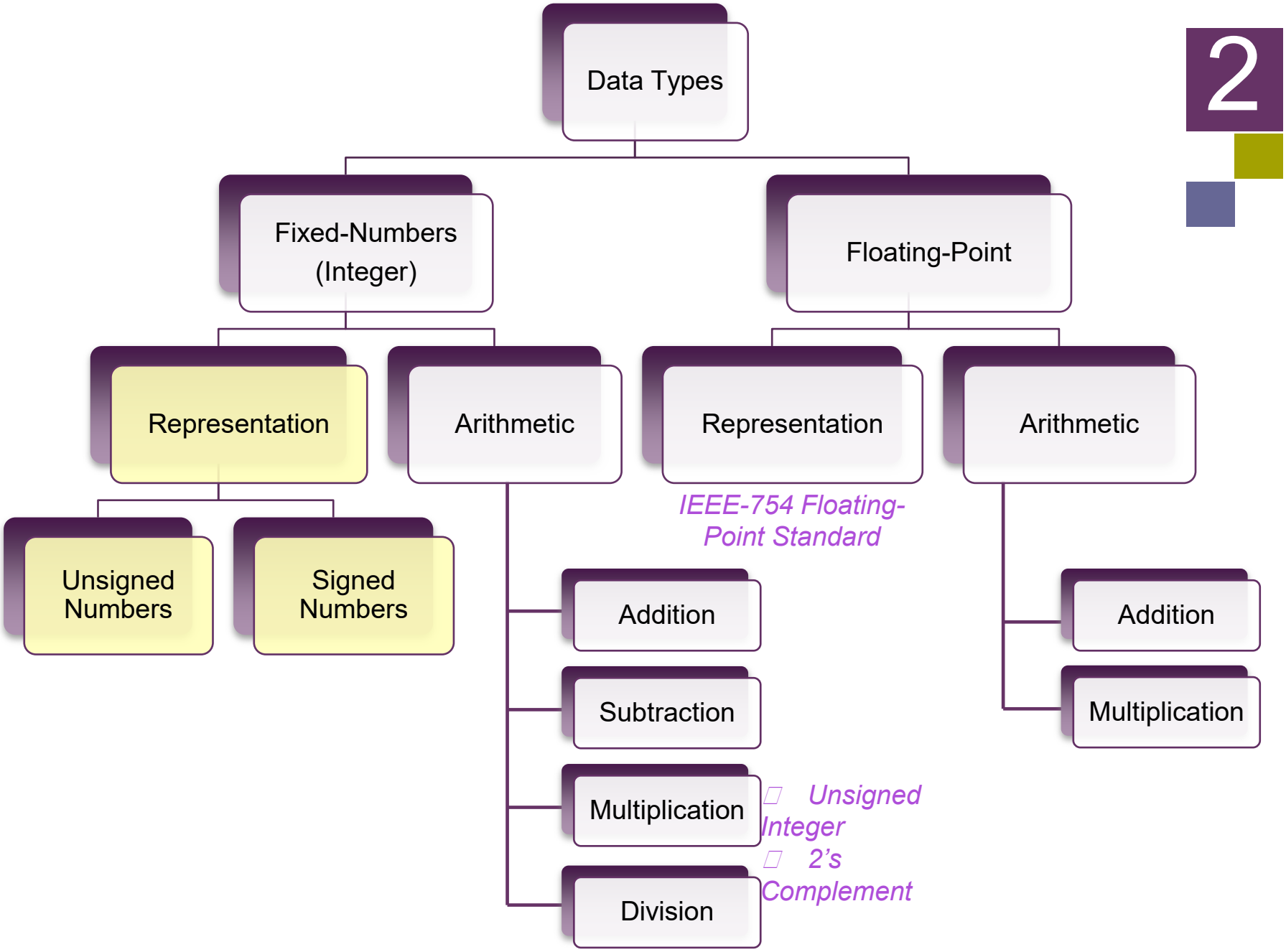
- *How are negative numbers represented?*
- *What is the largest number that can be represented in a computer world?*
- *What happens if an operation creates a number bigger than can be represented?*
- *What about fractions and real numbers?*
- *A mystery: How does hardware really multiply or divide numbers?*



2

- We begin our examination of the processor with an overview of the arithmetic and logic unit (ALU) → *computer arithmetic*.
- Computer arithmetic is commonly performed on two very different types of numbers: integer and floating point.

- In both cases, the representation chosen is a crucial design issue and is treated first, followed by a discussion of arithmetic operations.



William Stallings (2013). *Computer Organization and Architecture: Designing for Performance* (9th Edition). United States: Pearson Education Limited, p.341.
Linda Null and Julia Lobur (2003). *The Essentials of Computer Organization and Architecture.* United States: Jones and Bartlett Publishers. p.37.

The Arithmetic and Logic Unit

- The ALU is that part of the computer that actually performs arithmetic and logical operations on data.
- All electronic components in the computer are based on the use of simple digital logic devices that can store binary digits and perform simple Boolean logic operations.

- Data are presented to the ALU in registers,
- and the results of an operation are stored in registers.

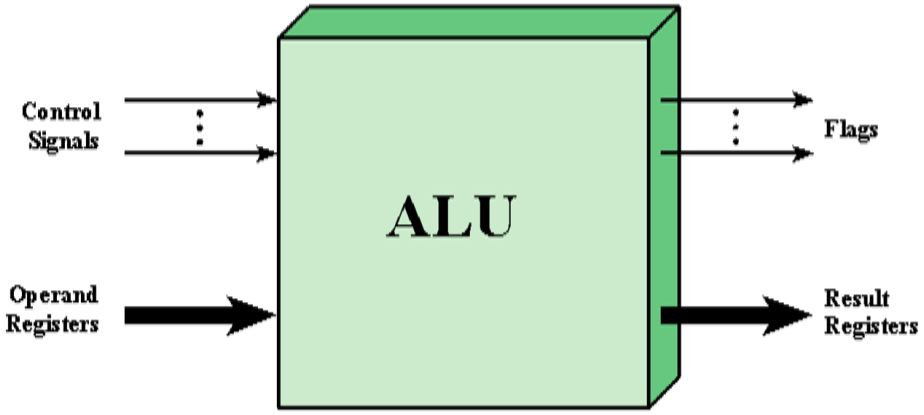


Figure : ALU inputs and outputs.

Module 2

Data Representation in Computer Systems



2.1 Introduction

2.2 Fixed-Number (Integer) Representation

- Overview
- Unsigned Numbers
- Signed Numbers

2.3 Fixed-Number (Integer) Arithn

2.4 Floating-Points Representatio

2.5 Floating-Points Arithmetic

2.6 Summary

RECAP

2

Overview

(Binary Number)

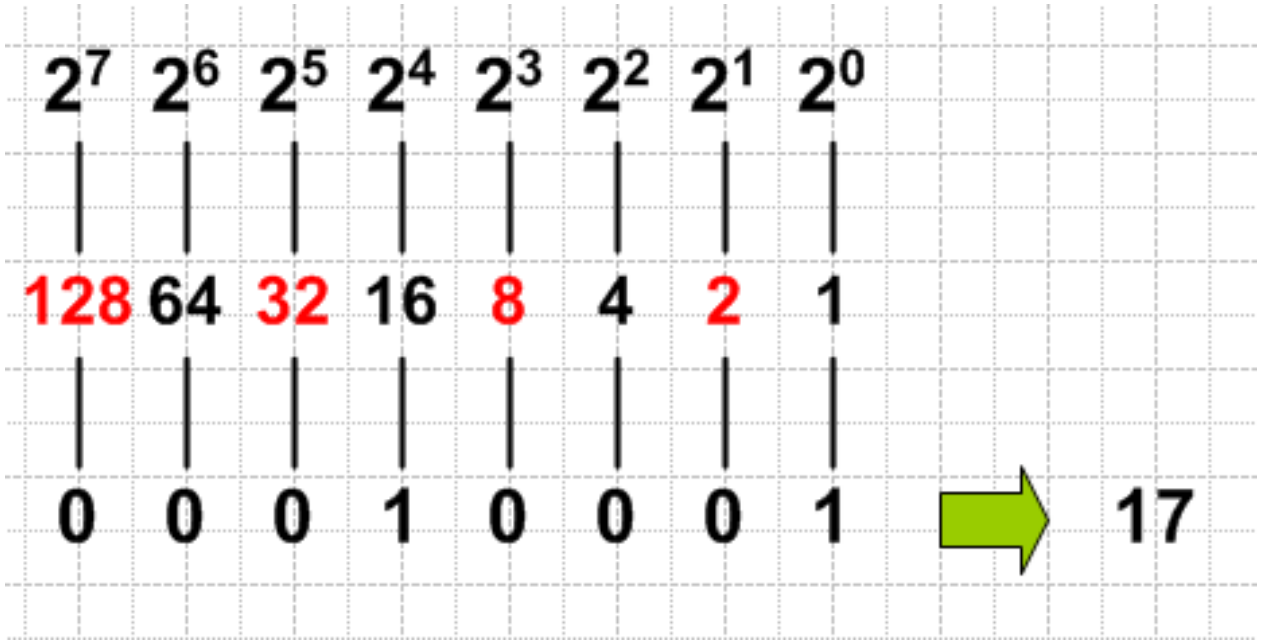
- Numbers are kept in computer hardware as a series of high and low electronic signals.
- They are considered base 2 numbers (Binary)
- All information is composed of binary digits or *bits*.
- In any number base, the value of i th digit d is

$$d \times Base^{i}$$

where i start at 0 and increases from right to left.

RECAP

- Base 2



$$d \times Base^{i}$$

RECAP

2

Why nibble ?
... related to hex number 👉

Binary Equivalents

- 1 *Nybble* (or *nibble*) = 4 *bits*
- 1 *Byte* = 2 *nybbles* = 8 *bits*
- 1 *Kiloyte* (*KB*) = 1024 *Bytes*
- 1 *Megabyte* (*MB*) = 1024 *Kilobytes* = 1,048,576 *Bytes*
- 1 *Gigabyte* (*GB*) = 1024 *Megabytes* = 1,073,741,824 *Bytes*

RECAP

Hexadecimal Numbers

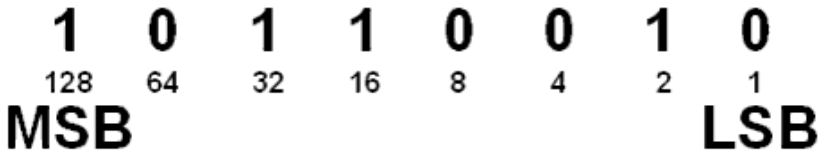
2

- Hexadecimal (or hex in short) system is a *short-hand* for binary system.
- It *groups 4 bits* into one entity.
- The hex numbers go from 0 to 9, then A, B, C, D, E, F.

decimal	hexadecimal	binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

RECAP

Size of Binary Numbers

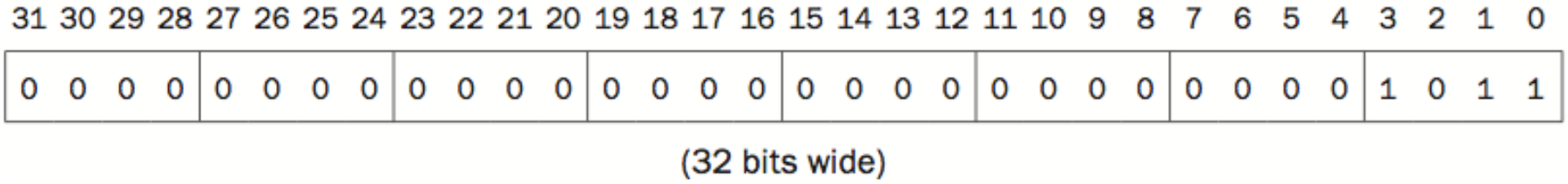


- 8 bits long
- Range of numbers that can be represented;
- = 0 to $(2^8 - 1)$ = 0 to 255

RECAP

Size of Binary Numbers

2



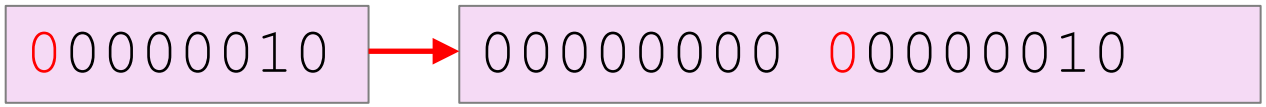
- 32 bits long
- Range of numbers that can be represented =
- = 0 to $(2^{32} - 1)$ = 0 to 4,294,967,295

RECAP

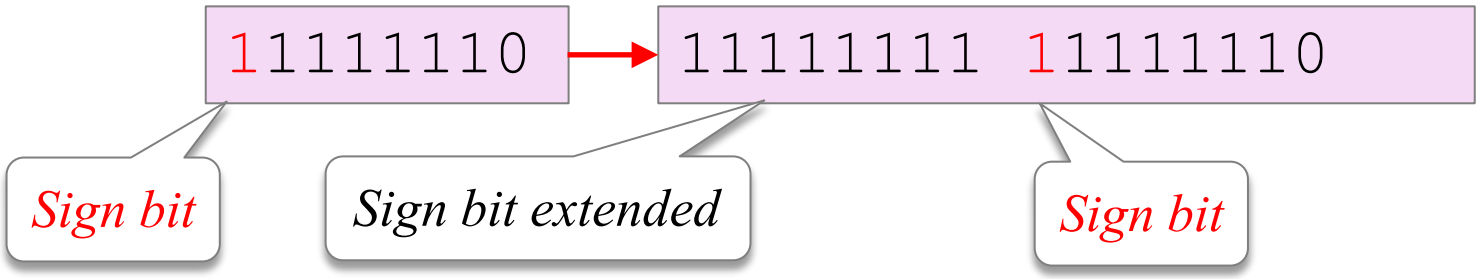
2 Size of Binary Numbers

(Sign Extension)

- Extending a number representation to a larger number of bits.
- **Example**: 2 in 8 bit binary to 16 bit binary.



- In signed numbers, it is important to extend the sign bit to *preserve* the number (+ve or –ve)
- **Example**: (–2) in 8 bit binary to 16 bit binary.



RECAP

2

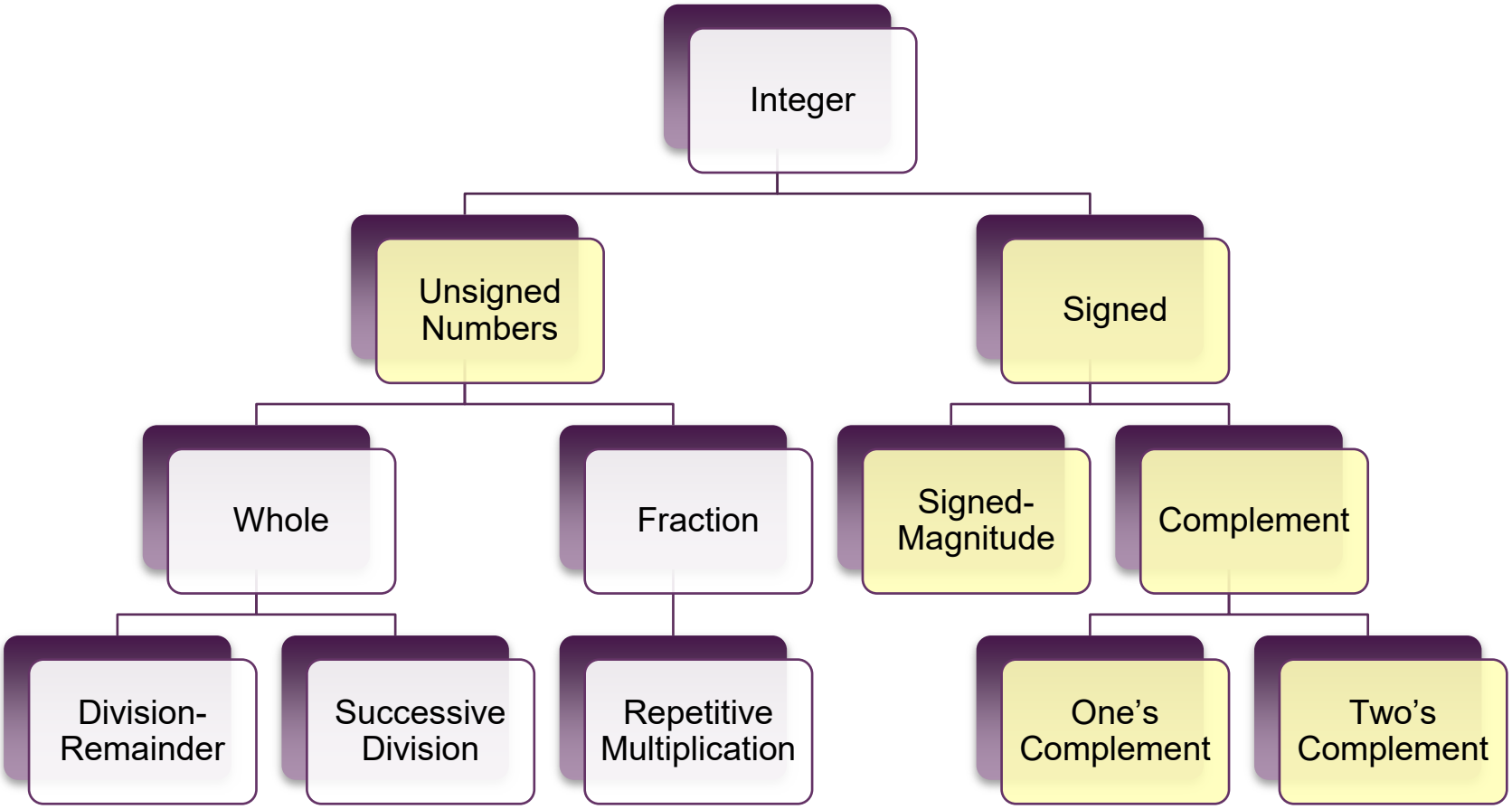


Figure: Types of numbers.

RECAP

Unsigned Numbers

2

- For positive number only.
- Range number = 0 to 2^n-1
- *There is no negative values representation*.

If 32 bits long.
Range of numbers that can be represented
= 0 to (2^{32} – 1)
= 0 to 4,294,967,295

RECAP

2

MIPS (Millions Instruction Per Second)
LSB (Least Significant Bit)
MSB (Most Significant Bit)

- Example, MIPS word is 32 bits long:
  - LSB → bit 0
  - MSB → bit 31

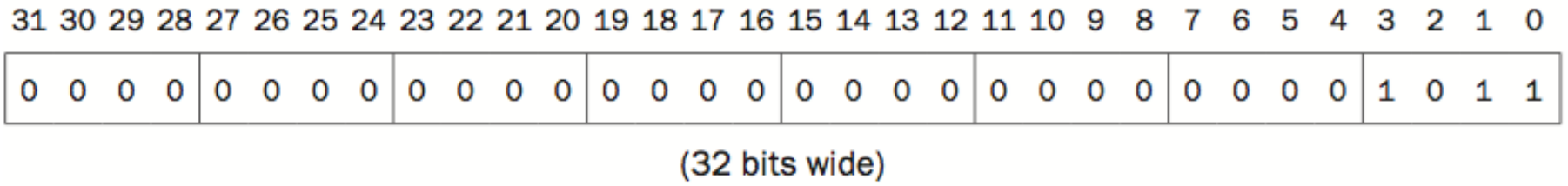


Figure: The numbering of bits in MIPS word for 1011_2

- Range Number can represent : 0 – $4{,}294{,}967{,}295_{10}$

MIPS Architecture

2

MIPS (originally an acronym for **Microprocessor without Interlocked Pipeline Stages**) is a reduced instruction set computer (RISC) instruction set architecture (ISA) developed by MIPS Technologies (formerly MIPS Computer Systems, Inc.). The early MIPS architectures were 32-bit, with 64-bit versions added later. Multiple revisions of the MIPS instruction set exist, including MIPS I, MIPS II, MIPS III, MIPS IV, MIPS V, MIPS32, and MIPS64. The current revisions are MIPS32 (for 32-bit implementations) and MIPS64 (for 64-bit implementations).[1][2] MIPS32 and MIPS64 define a control register set as well as the instruction set.

Several optional extensions are also available, including MIPS-3D which is a simple set of floating-point SIMD instructions dedicated to common 3D tasks,[3] MDMX (MaDMaX) which is a more extensive integer SIMD instruction set using the 64-bit floating-point registers, MIPS16e which adds compression to the instruction stream to make programs take up less room,[4] and MIPS MT, which adds multithreading capability.[5]

Computer architecture courses in universities and technical schools often study the MIPS architecture.[6] The architecture greatly influenced later RISC architectures such as Alpha.

MIPS

Designer	MIPS Technologies, Inc.
Bits	64-bit (32→64)
Introduced	1981
Design	RISC
Type	Register-Register
Encoding	Fixed
Branching	Condition register
Endianness	Bi
Extensions	MDMX, MIPS-3D
Registers	
General purpose	31 (R0=0)
Floating point	32 (paired DP for 32-bit)

MIPS implementations are primarily used in embedded systems such as Windows CE devices, routers, residential gateways, and video game consoles such as the Sony PlayStation 2 and PlayStation Portable. Until late 2006, they were also used in many of SGI's computer products. MIPS implementations were also used by Digital Equipment Corporation, NEC, Pyramid Technology, Siemens Nixdorf, Tandem Computers and others during the late 1980s and 1990s. In the mid to late 1990s, it was estimated that one in three RISC microprocessors produced was a MIPS implementation.[7]

RECAP

Signed Numbers

- Integers as binary values can be positive or negative.
- Problem:
  - *How to represent and encode the actual sign of a number?*
- Solution:
  - Need code to represent the signs.
- Three signed representations covered:
  - Signed Magnitude
  - Ones Complement
  - Twos Complement

RECAP

2

Signed Numbers

(a) Signed-Magnitude

- The conversions we have so far presented have involved only positive numbers.
- To represent negative values, computer systems allocate the high-order bit to indicate the sign of a value.
  - The high-order bit is the leftmost bit in a byte.
  - It is also called the Most Significant Bit (MSB).
- The remaining bits contain the value of the number.

RECAP

2

Example 1:

Add 79_{10} to 35_{10} using signed-magnitude arithmetic in 8-bit binary.

			1	1	1	1		⇐ carries	
0		1	0	0	1	1	1	1	(79)
0	+	0	1	0	0	0	1	1	+ (35)
0		1	1	1	0	0	1	0	(114)

7 bits

0 represents positive

RECAP

2

- The sum of two positive numbers, which is positive.
- ***Overflow*** (and thus an erroneous result) in signed numbers occurs when the sign of the result is **incorrect** .
- The *sign bit* is used only for the sign, so we can't "carry into" it, otherwise the result will be truncated as the MSB bit overflows, giving an *incorrect sum*.



- If the *overflow* bit is not discard, it would carry into the sign, causing the more outrageous result of the sum of two positive numbers being negative.

Example 2:

Add 01001111_2 to 01100011_2 using signed-magnitude arithmetic in 8-bit binary.

Last carry	1	←			1	1	1	1	⇐ carries
overflows and is	0		1	0	0	1	1	1	1 (79)
discarded.	0	+	1	1	0	0	0	1	1 + (99)
	0		0	1	1	0	0	1	0 (50)

Error result !

Answer should be 178_{10}

Example 3:

Add (-19_{10}) to 13_{10} using signed-magnitude arithmetic in 8-bit binary.

- The first number (*augend*) is negative (sign bit 1)
- The second number (*addend*) is positive (sign bit 0)
- Since larger magnitude is *augend*, then subtract to *addend*.

$$\begin{array}{cccccccccc} & & & 0 & 1 & 2 & & & \Leftarrow \text{borrows} \\ 1 & & 0 & 0 & \not{1} & \not{0} & \not{0} & 1 & 1 & (-19) \\ 0 & - & 0 & 0 & 0 & 1 & 1 & 0 & 1 & +(13) \\ \hline 1 & & 0 & 0 & 0 & 0 & 1 & 1 & 0 & (-6) \end{array}$$

The sign of result will be same as sign of larger magnitude.

RECAP

2

- In signed magnitude, the sign bit is used only for the sign.



Problem 1:

If there any carry emitting from the last bit, the result will be truncated as the last bit overflow, giving an incorrect sum.



Problem 2:

Complicated to define the larger magnitude, to subtract negative number, borrow from the *minuend*.

Solution : Need a simpler method for representing signed numbers → complement systems.

RECAP

2

Signed Numbers

(b) One's Complement

- The **MSB** is the sign bit
- Negative numbers are obtained by flipping (or *complementing*) the bit → **0 to 1**; **1 to 0**

Example:

$+25_{10} = 0001\ 1001_2$

$-25_{10} = 1110\ 0110_2$

- *Note that this leads to having **two representations** for the number zero*
- With an 8-bit number representation:
  - Largest number is +127,
  - Smallest number is -127

In 8 bits numbers:
0000 0000 = +0
0111 1111 = +127
1111 1111 = -0
1000 0000 = -127

Example 4:

Using one's complement 8-bit binary arithmetic, find the sum of 9_{10} and (-23_{10}).

The last carry is zero so we are done.	$0 \leftarrow$	$0\ 0\ 0\ 0\ 1\ 0\ 0\ 1$	(9)
		$+\ 1\ 1\ 1\ 0\ 1\ 0\ 0\ 0$	$+\ (-23)$
		$1\ 1\ 1\ 1\ 0\ 0\ 0\ 1$	-14_{10}

00010111 (*23*)
11101000 (*1s*)

RECAP

2

Example 5:

Using one's complement 8-bit binary arithmetic, find the sum of (-9_{10}) and 23_{10}.

	$1 \leftarrow 1\ 1\ 1\quad 1\ 1$	$\Leftarrow$ carries
	$0\ 0\ 0\ 1\ 0\ 1\ 1\ 1$	(23)
The last	$+\ 1\ 1\ 1\ 1\ 0\ 1\ 1\ 0$	$\underline{+(-9)}$
carry is added	$0\ 0\ 0\ 0\ 1\ 1\ 0\ 1$	
	$+\ 1$	
to the sum.	$0\ 0\ 0\ 0\ 1\ 1\ 1\ 0$	14_{10}

00001001 (9)
11110110 (*1s*)

With one's complement addition, the carry bit is "carried around" and added to the sum.

RECAP

2

Signed Numbers

(c) Two's Complement

- The **MSB** is the sign bit
- Negative numbers are obtained by *adding 1* to *One's complement negative*.
- *Note that this has only **one representation** for the number zero*
- With an 8-bit number representation:
  - Largest number is +127,
  - Smallest number is -128.

In 8 bits numbers:
0000 0000 = +0
0111 1111 = +127
1111 1111 = -128
1000 0000 = -127

RECAP

2

Signed Numbers

Conversion of two's Complement

- **Step 1:**
  - do one's complement (flip the bits → **0 to 1**; **1 to 0**)
- **Step 2:**
  - then add one.
- Example: -17

17 = 00010001

Step1 : 11101110

Step2 : 11101110
+ 1

11101111

-17 = 11101111

RECAP

2

Example 6:

Express 23_{10}, (-23_{10}), and (-9_{10}) in 8-bit binary two's complement form.

$$
\begin{aligned}
23_{10} &= +(00010111_2) = 00010111_2 \\
-23_{10} &= -(00010111_2) = 11101000_2 + 1 = 11101001_2 \\
-9_{10} &= -(00001001_2) = 11110110_2 + 1 = 11110111_2
\end{aligned}
$$

Example 7:

Add 9_{10} to (-23_{10}) using 8-bit binary two's complement arithmetic.

$$
\begin{array}{rr}
0\ 0\ 0\ 0\ 1\ 0\ 0\ 1 & (9) \\
+\ 1\ 1\ 1\ 0\ 1\ 0\ 0\ 1 & +(-23) \\
\hline
1\ 1\ 1\ 1\ 0\ 0\ 1\ 0 & -14_{10}
\end{array}
$$

Example 8:

Find the sum of 23_{10} and (-9_{10}) in binary using two's complement arithmetic .

	$1\leftarrow$	1	1	1		1	1	1	$\Leftarrow$ carries
Discard		0	0	0	1	0	1	1	1	(23)
carry.	+	1	1	1	1	0	1	1	1	+ (–9)
		0	0	0	0	1	1	1	0	14_{10}

With two's complement addition, the carry bit is discarded.

Aside: Detecting Overflow

2

- Notice that the discarded carry in Example 8 did not cause an erroneous result.
- An overflow occurs if:
  - two positive numbers are added and the result is negative, or
  - two negative numbers are added and the result is positive.
- It is not possible to have overflow when using *two's complement notation* if a positive and a negative number are being added together.

2

- A simple rule for detecting an overflow condition using two's complement arithmetic (Assume 8-bit binary):

If the carry into the *sign bit* equals the carry out of the bit, no overflow has occurred.

	1 ←	1	1		1	1	1		⇐ carries
Discard		0	0	0	1	0	1	1	1	(23)
carry.	+	1	1	1	1	0	1	1	1	+ (–9)
		0	0	0	0	1	1	1	0	14_{10}

If the carry into the *sign bit* is different from the carry out of the sign bit, overflow (and thus an error) has occurred.

	0 ←	1	1	1					⇐ carries
Discard last		0	1	1	1	1	1	1	0	(126)
carry.	+	0	0	0	0	1	0	0	0	+(8)
		1	0	0	0	0	1	1	0	(–122???)

Module 2

Data Representation in Computer Systems

2.1 Introduction
2.2 Fixed-Number (Integer) Representation

2.3 Fixed-Number (Integer) Arithmetic

2.4 Floating-Points Representation
2.5 Floating-Points Arithmetic
2.6 Summary

- Negation
- Addition
- Subtraction
- Multiplication
- Division

2.3 Fixed-Number Arithmetic

2

- This section examines common arithmetic functions on numbers in two's complement representation.

Negation

- In two's complement notation, the *negation* of an integer can be formed with the following rules:

1) Take the Boolean complement of each bit of the integer (including the sign bit) → Set each 1 to 0 and each 0 to 1.
2) Treating the result as an unsigned binary integer, add 1.

- Those two-step process is referred to as the two's complement operation, or the taking of the two's complement of an integer.

$$
\begin{array}{rcll}
+18 & = & 00010010 & \text{(twos complement)} \\
\text{bitwise complement} & = & 11101101 & \\
 & & \underline{+\qquad 1} & \\
 & & 11101110 & = -18
\end{array}
$$

$$
\begin{array}{rcll}
-18 & = & 11101110 & \text{(twos complement)} \\
\text{bitwise complement} & = & 00010001 & \\
 & & \underline{+\qquad 1} & \\
 & & 00010010 & = +18
\end{array}
$$

2

(a) Addition

- Digits are added bit by bit from right to left, with carries passed to the next digit to the left.

RECAP

Rules of Binary Addition

- 0 + 0 = 0 carry = 0
- 0 + 1 = 1 carry = 0
- 1 + 0 = 1 carry = 0
- 1 + 1 = 0 carry = 1

Example 9:

Addition of numbers in two's complement representation.

1001 = −7 +0101 = 5 1110 = −2 (a) (−7) + (+5)	1100 = −4 +0100 = 4 10000 = 0 (b) (−4) + (+4)
0011 = 3 +0100 = 4 0111 = 7 (c) (+3) + (+4)	1100 = −4 +1111 = −1 11011 = −5 (d) (−4) + (−1)

Discard !

Discard !

(e) and (f) show examples of overflow that can occur whether or not there is a carry.

0101 = 5 +0100 = 4 1001 = Overflow (e) (+5) + (+4)	1001 = −7 +1010 = −6 10011 = Overflow (f) (−7) + (−6)

2 (b) Subtraction

- Subtraction is easily handled with the following rule:

- **SUBTRACTION RULE:** To subtract one number (*subtrahend*) from another (*minuend*), take the two's complement (*negation*) of the *subtrahend* and add it to the *minuend*.

M (Minuend)
S (Subtrahend)

$$M - S = M + (-S)$$

(–S) → Two's Complement (negation)

Example 10:

Subtraction of numbers in two's complement representation $(M - S)$.

$$M - S = M + (-S)$$
$$= 2 + (-7)$$

M (Minuend)
S (Subtrahend)

```
        0010 =  2
       +1001 = –7
        1011 = –5

(a) M =  2 = 0010
    S =  7 = 0111
   –S =       1001
```

```
        0101 =  5
       +1110 = –2
       10011 =  3

(b) M =  5 = 0101
    S =  2 = 0010
   –S =       1110
```

```
        1011 = –5
       +1110 = –2
       11001 = –7

(c) M = –5 = 1011
    S =  2 = 0010
   –S =       1110
```

```
        0101 = 5
       +0010 = 2
        0111 = 7

(d) M =  5 = 0101
    S = –2 = 1110
   –S =       0010
```

Overflow !

```
        0111 = 7
       +0111 = 7
        1110 = Overflow

(e) M =  7 = 0111
    S = –7 = 1001
   –S =       0111
```

```
        1010 = –6
       +1100 = –4
       10110 = Overflow

(f) M = –6 = 1010
    S =  4 = 0100
   –S =       1100
```

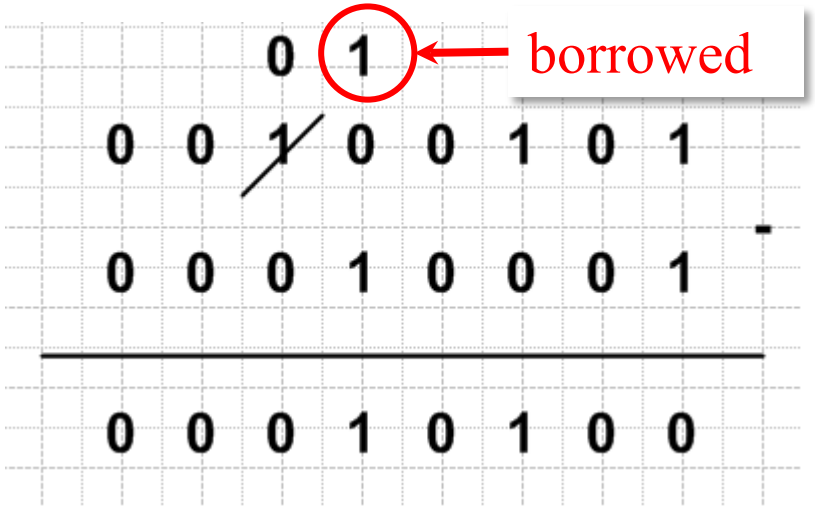
- Subtraction can be done directly.
- Rules of binary subtraction:

$0 - 0 = 0$
$0 - 1 = 1$, *and borrow 1 from the next more significant bit*
$1 - 0 = 1$
$1 - 1 = 0$

Example 11:

$37_{10} - 17_{10}$

(Assume 8 bit binary)



(c) Multiplication

- Multiplication is a complex operation, whether performed in hardware or software.
- The simpler problem of multiplying using unsigned integers, and the most common techniques for multiplication of numbers is two's complement representation.
- Multiplication of binary numbers must always use 0 and 1.

Rules of Binary Multiplication:

0 x 0 = 0
0 x 1 = 0
1 x 0 = 0
1 x 1 = 1
and no carry or borrow bit.

2

(c) Multiplication

Unsigned Integer

- Consist of operands called *multiplicand* and *multiplier*, and final result as *product.*

```
      Multiplicand
x     Multiplier
-------------------
      Product
-------------------
```

Example 12:

Multiply the unsigned binary numbers of 1011_2 by 1101_2.

1011_2 → (11) *Multiplicand*

x 1101_2 → (13) *Multiplier*

1011

0000

1011

1011 .

10001111_2 → (143) *Product*

Partial products

If we ignore the sign bits, the length of multiplication of an n-bit *multiplicand* and an m-bit *multiplier* is a *product* that is $(n+m)$ bit long.

(c) Multiplication

Signed Integer: Two's Complement

- We have seen that addition and subtraction can be performed on numbers in two's complement notation by treating them as unsigned integers.
- **Example**:

$$\begin{array}{r} 1001 \\ +\ 0011 \\ \hline 1100 \end{array}$$

If these numbers are considered to be unsigned integers, then we are adding 9 (1001_2) plus 3 (0011_2) to get 12 (1100_2)

As two's complement integers, we are adding –7 (1001_2) to 3 (0011_2) to get –4 (1100_2).

- Unfortunately, the previous simple scheme will not work for multiplication.

Example 13:

To see this, consider again **Example 12**.

Multiply the two's complement binary numbers of 1011_2 by 1101_2.

$$
\begin{array}{rll}
 & 1011_2 & \rightarrow (-5)\ \textit{Multiplicand} \\
\times & \underline{1101_2} & \rightarrow (-3)\ \textit{Multiplier} \\
 & 1011 & \\
 & 0000 & \\
 & 1011 & \\
 & \underline{1011} & \\
 & 10001111_2 & \rightarrow (-113)\ \textit{Product}
\end{array}
$$

Partial products

- The example demonstrates that straightforward multiplication will not work if both the multiplicand and/or multiplier are negative.



Regular multiplication clearly yields incorrect result !

- **Solution** : Multiplication algorithm.

  This algorithm has the benefit of speeding up the multiplication process, relative to a more straightforward approach.

A Multiplication Algorithm and Hardware

2

- Multiplication must cope with overflow because we frequently want a 32-bit *product* as the result of multiplying two 32-bit numbers.

- In the next slides, assume that we are multiplying only positive number (unsigned) with the 1st version of highly optimized multiplication hardware.

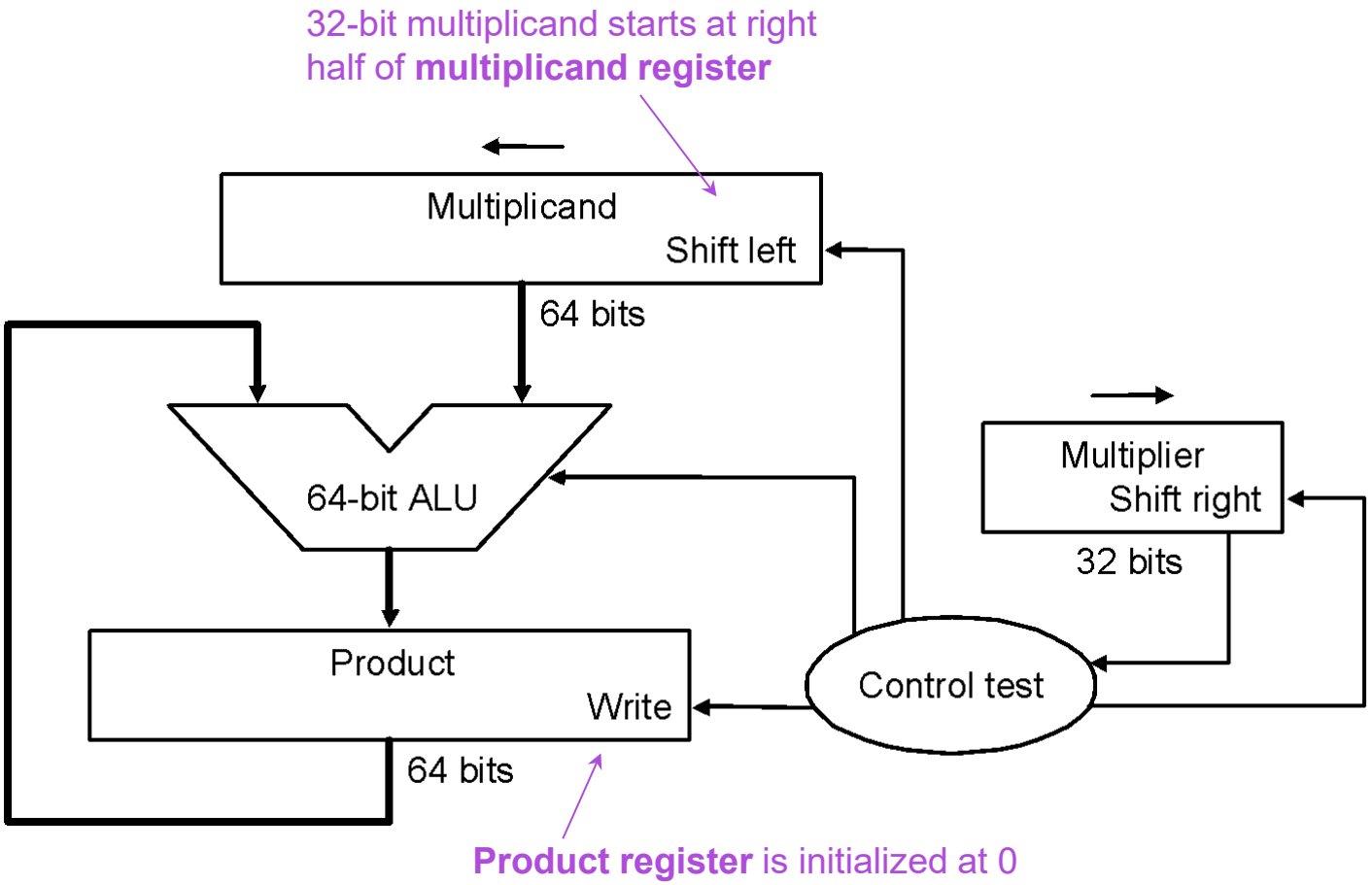


Figure: First version of Multiplication Hardware.

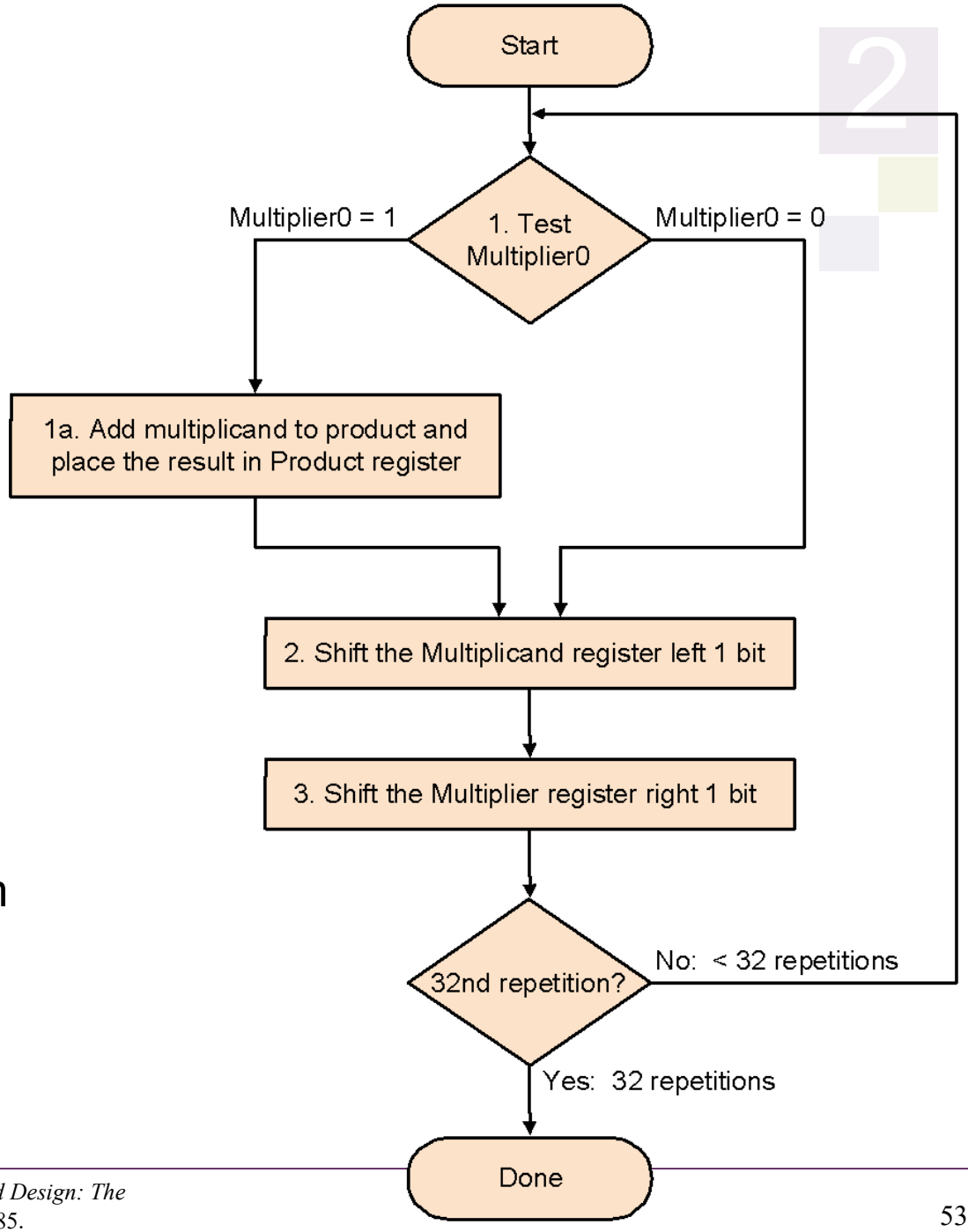


Figure:
The Multiplication Algorithm using the Hardware.

Example 14:

Using 4-bit numbers, multiply of 2_{10} x 3_{10} .

2 x 3 = ?

Product (P)

0010_2 x 0011_2

Multiplier (MP)

Multiplicand (MC)

Steps:

1 – Test multiplier (0 or 1)

If **1** then 1a: P = P + MC

If **0** then no operation

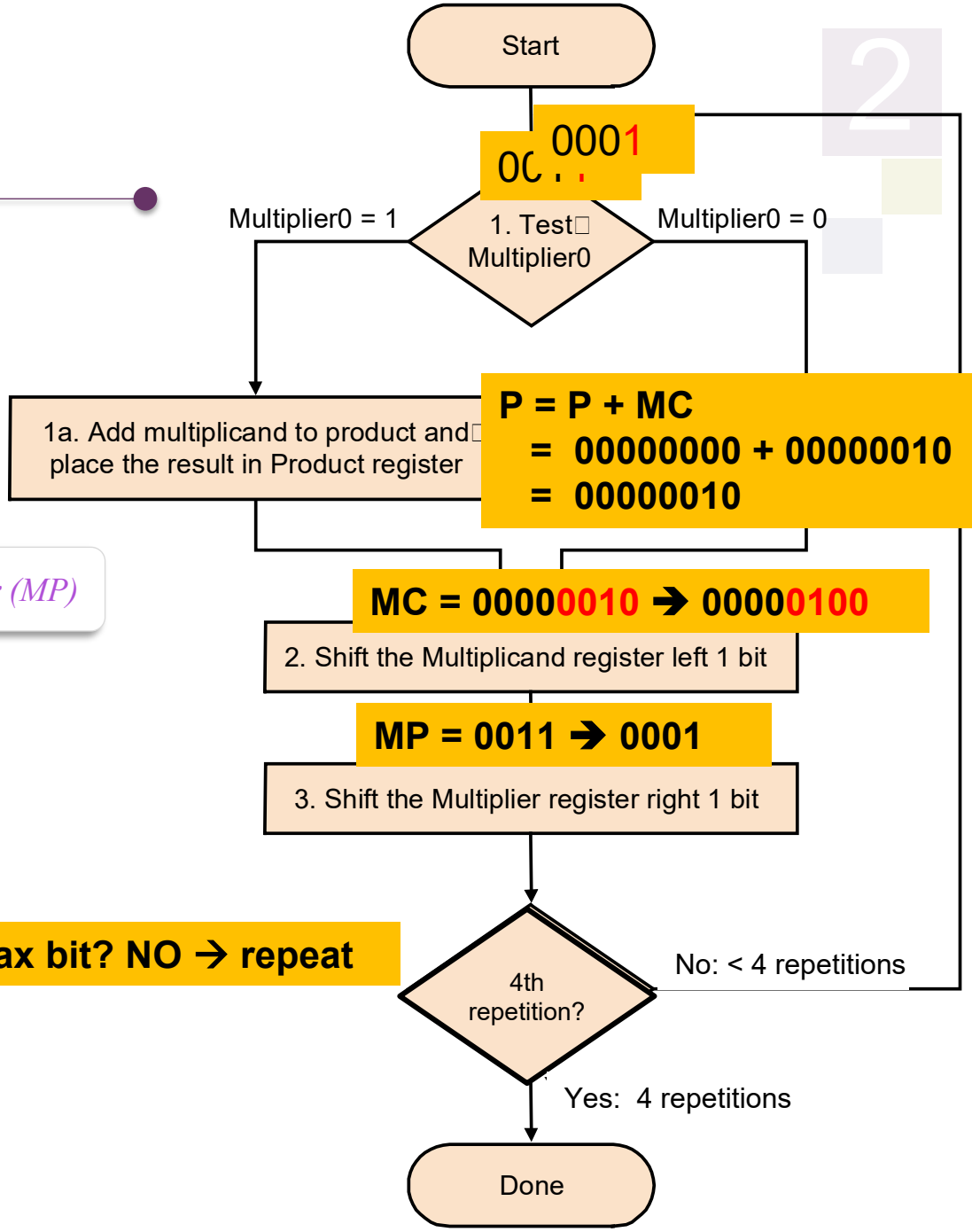
2 – Shift MC left

3 – Shift MP right

All bits done?

If still < max bit, repeat

If = max bit, stop



Max bit? NO → repeat

2 x 3 (0010 x 0011)

Iteration	Step	Multiplier (MP)	Multiplicand (MC)	Product (P)
0	Initial value	0011	0000 0010	0000 0000
1	1a:1➔P = P + MC			0000 0010
	2: Shift MC left		0000 0100	
	3: Shift MP right	0001		
2	1a:1➔P = P + MC			0000 0110
	2: Shift MC left		0000 1000	
	3: Shift MP right	0000		6
3	1a:0➔no operation			
	2: Shift MC left		0001 0000	
	3: Shift MP right	0000		
4	1a:0➔no operation			
	2: Shift MC left		0010 0000	
	3: Shift MP right	0000		

Aside:

- The *multiplier* (MP) must always in positive number.
- If involve negative number, do *additive inverse* to the *multiplicand* (MC) and the *multiplier* (MP).
- **Examples**:

$$7 \times (-5) = (-7) \times 5$$
$$(-7) \times (-5) = 7 \times 5$$

$$Multiplicand \times (-Multiplier) = (-Multicand) \times Multiplier$$
$$MC \times (-MP) = (-MC) \times MP$$

Example 15:

Using a 4-bit binary arithmetic, multiply 2_{10} with (-3_{10}) using the 1st version of highly optimized multiplication hardware.

Solution:

→ $2 \times (-3)$

- Do an *additive inverse* to the multiplicand (MC) and the MP:

$$(-2) \times 3$$

- Perform the multiplication as usual.

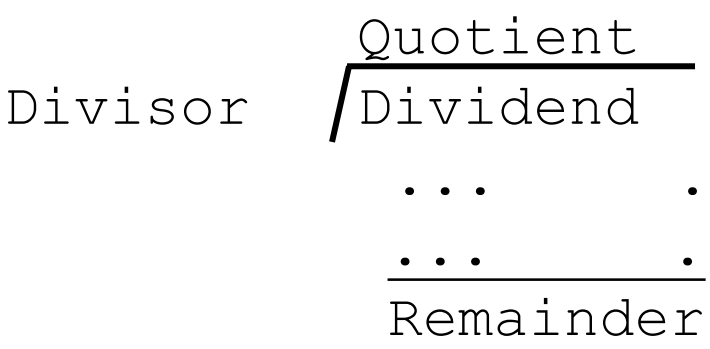
2 x (-3) ➔ get additive inverse of both

Iteration	Step	Multiplier (MP) 3	Multiplicand (MC) -2	Product (P)
0	Initial value	0011	1111 1110	0000 0000
1	1a:1➔P = P + MC			1111 1110
	2: Shift MC left		1111 1100	
	3: Shift MP right	0001		
2	1a:1➔P = P + MC			1111 1010
	2: Shift MC left		1111 1000	-6
	3: Shift MP right	0000		
3	1a:0➔no operation			
	2: Shift MC left		1111 0000	
	3: Shift MP right	0000		
4	1a:0➔no operation			
	2: Shift MC left		1110 0000	
	3: Shift MP right	0000		

(d) Division

- More complex than multiplication but is based on the same general principles.
- It even offers the opportunity to perform a mathematically invalid operations in dividing by 0.

- Two operands called *dividend* and *divisor*, the result as *quotient* with secondary result called *remainder*.



- Another way to express the relationship between the components:

$$Dividend = Quotient \times Divisor + Remainder$$

where the *remainder* is smaller than the *divisor*.

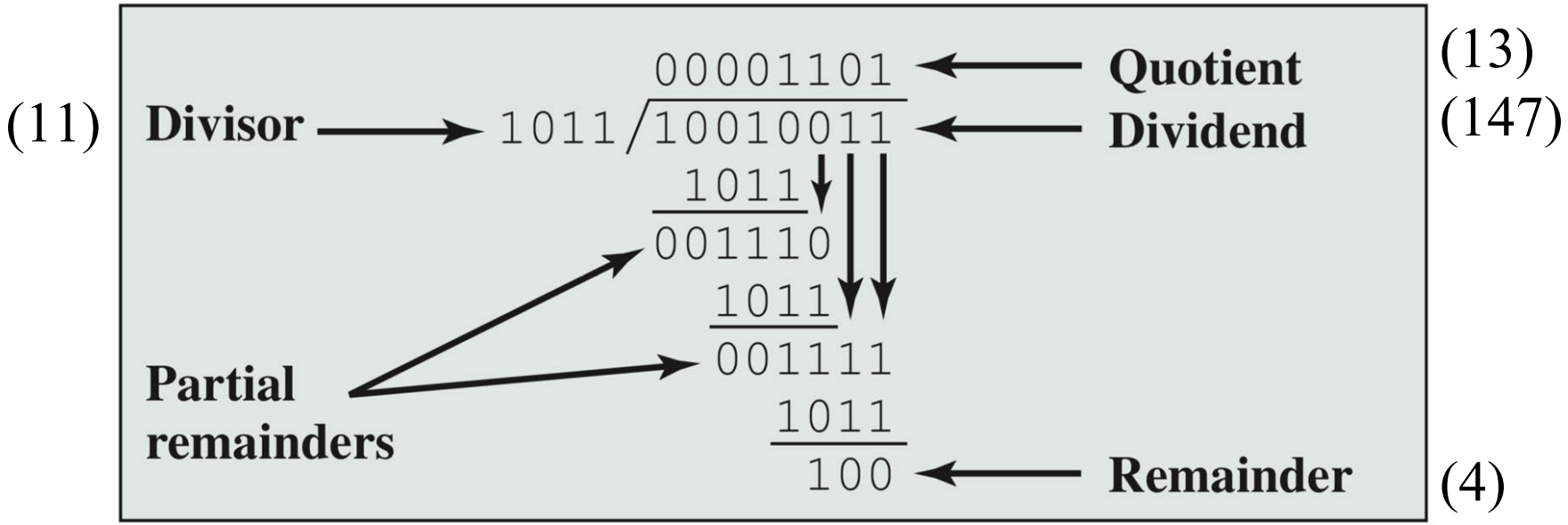
- Infrequently, programs use the divide instruction just to get the *remainder*, ignoring the *quotient*.

2

(d) Division

Unsigned Integer

- The following figure shows an example of the long division of unsigned binary integers of 147_{10} divided by 11_{10}.



(d) Division

Signed Integer: Two's Complement

- Make both *dividend* and *divisor* positive and perform division.
- Make the sign of the *remainder* match to the *dividend*, no matter what the signs of the *divisor* and *quotient*.
- The rules:

$$+7 \div +2: Quotient = +3, Remainder = +1$$
$$+7 \div -2: Quotient = -3, Remainder = +1$$
$$-7 \div +2: Quotient = -3, Remainder = -1$$
$$-7 \div -2: Quotient = +3, Remainder = -1$$

- *Negate* the *quotient* if *dividend* and *divisor* were of opposite signs.

Example 15:

Using 4-bit numbers, divide the two's complement binary numbers (-7_{10}) by 2_{10}.

$$
\begin{array}{lll}
 & 100 & (-4) \\
(+2)\quad 10\,\overline{)\,1001} & & (-7) \\
 & 10 & \\
 & \overline{00} & \\
 & 00 & \\
 & \overline{01} & \\
 & 00 & \\
 & \overline{1} &
\end{array}
$$

- Regular division clearly yields incorrect result;
- **Solution**: Division algorithm.

A Division Algorithm and Hardware

2

- Binary division is restricted to 0 or 1, thereby simplifying binary division.
- In the next slides, assume that both the *dividend* and *divisor* are positive number; Hence the *quotient* and *remainder* are non-negative.
- Since iteration of the algorithm needs to move the *divisor* to the right one digit, we start the *divisor* placed in the left half of the 64-bit Divisor Register.

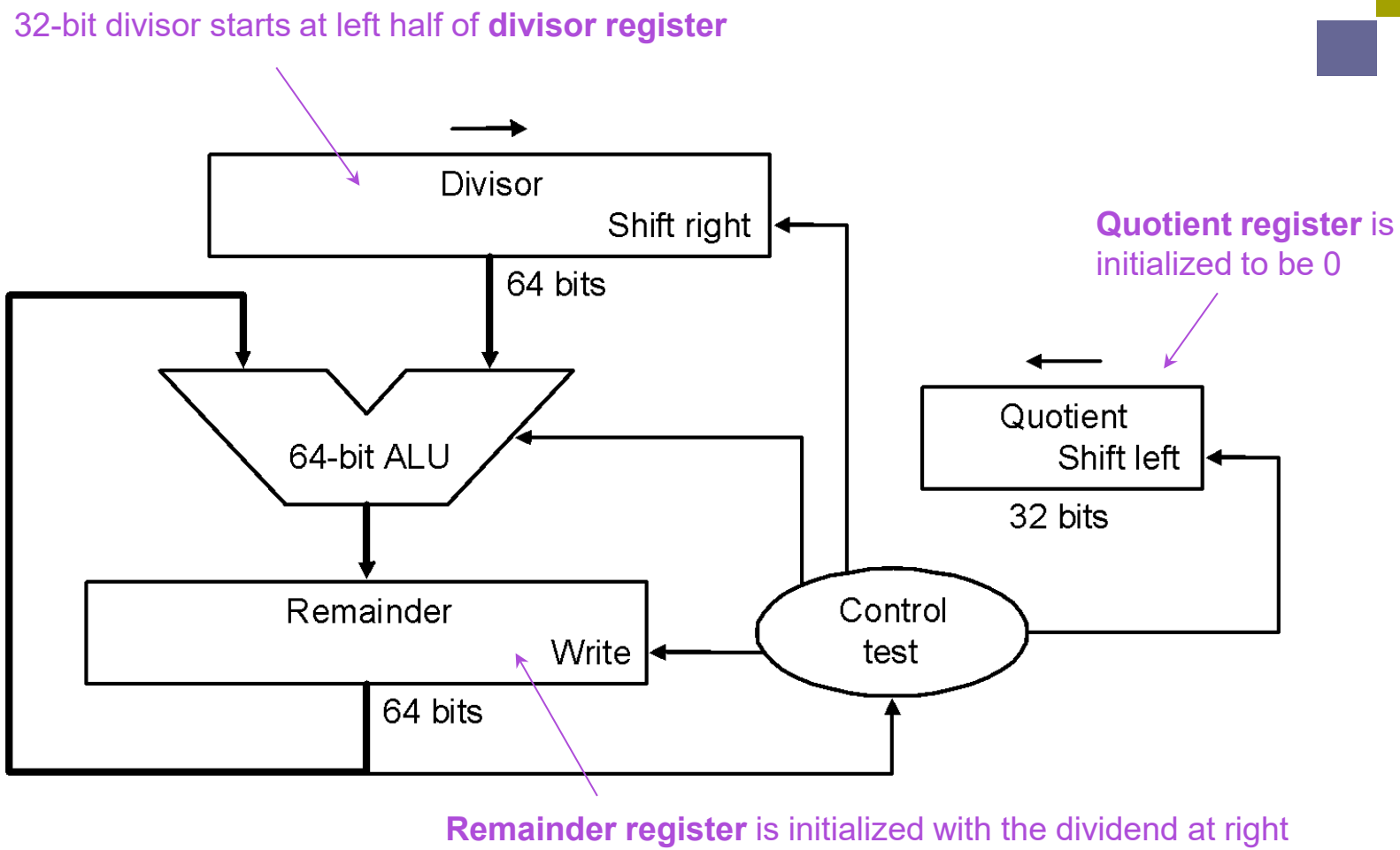


Figure: First version of Division Hardware.

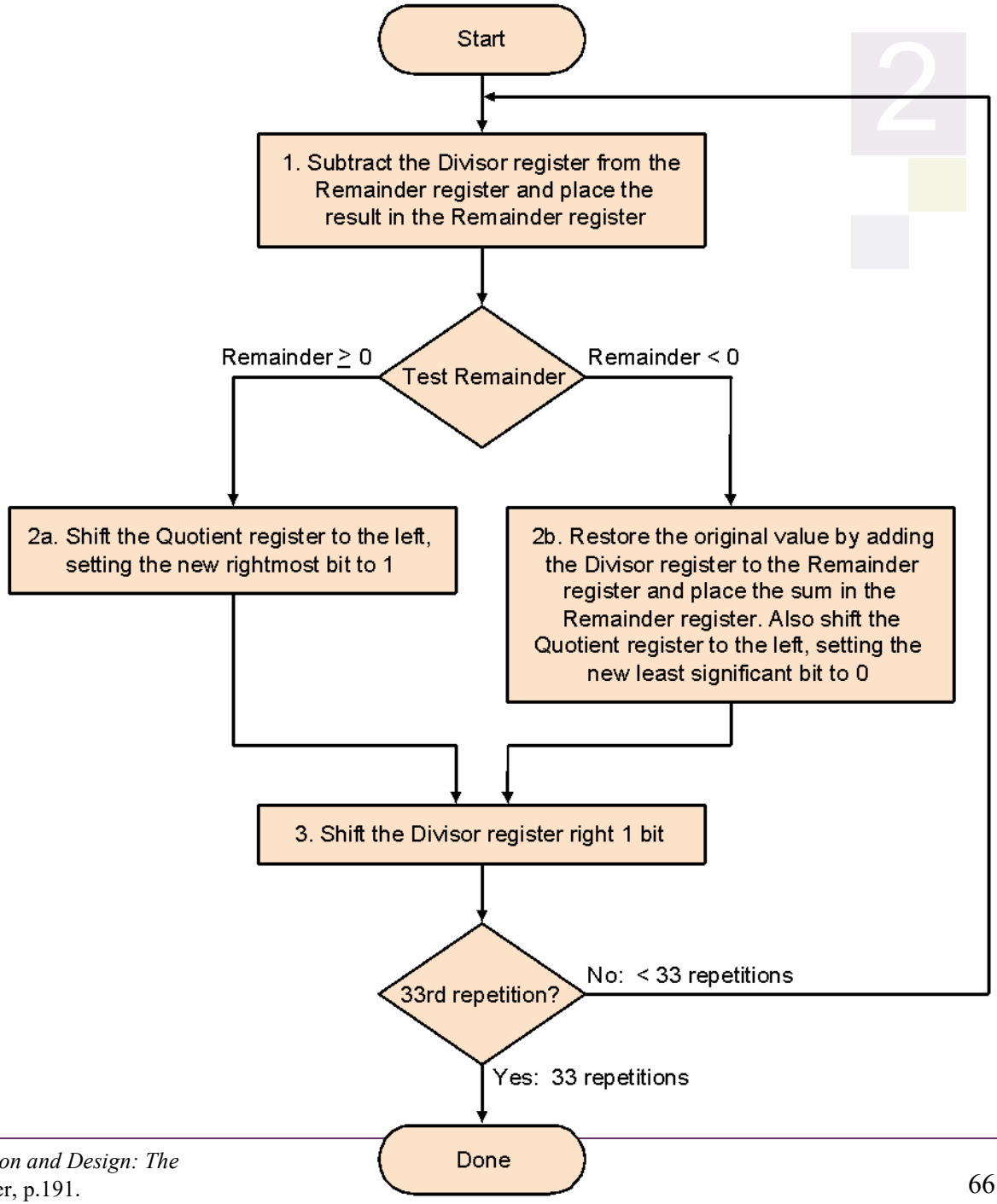
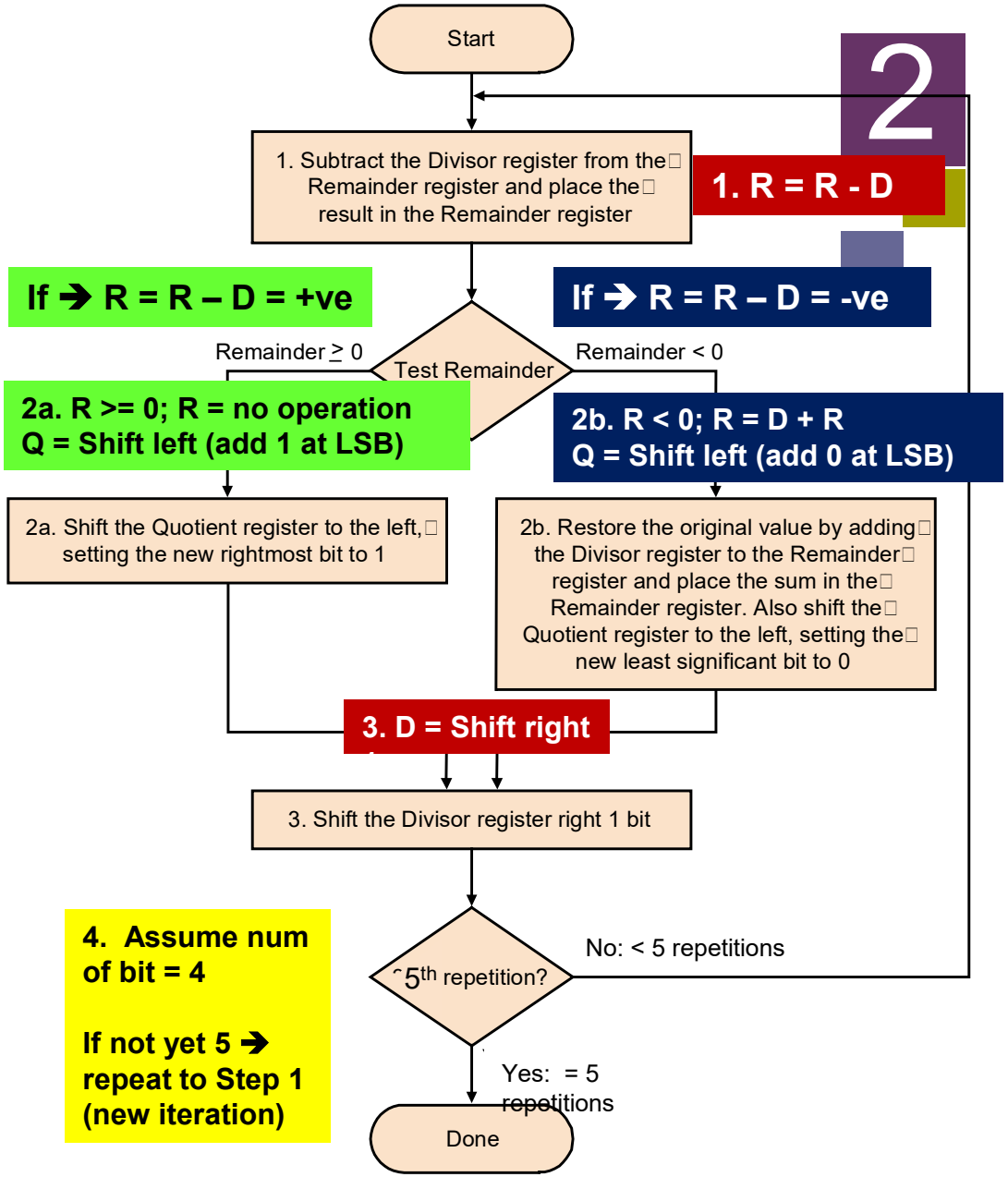


Figure:
The Division Algorithm using the Hardware.

Steps:

- 1 – Remainder (R) = R – D
- 2 – test new R (>= 0 or <0)
- 2a - If >=0 then
  - R = no operation;
  - Q = Shift left (add 1 at LSB)
- 2b - If <0 then
  - R = D + R
  - Q = Shift left (add 0 at LSB)
- 3 – shift D right
- 4 - All bits done?
  - If still < (max bit + 1) , repeat
  - If = (max bit+1) , stop



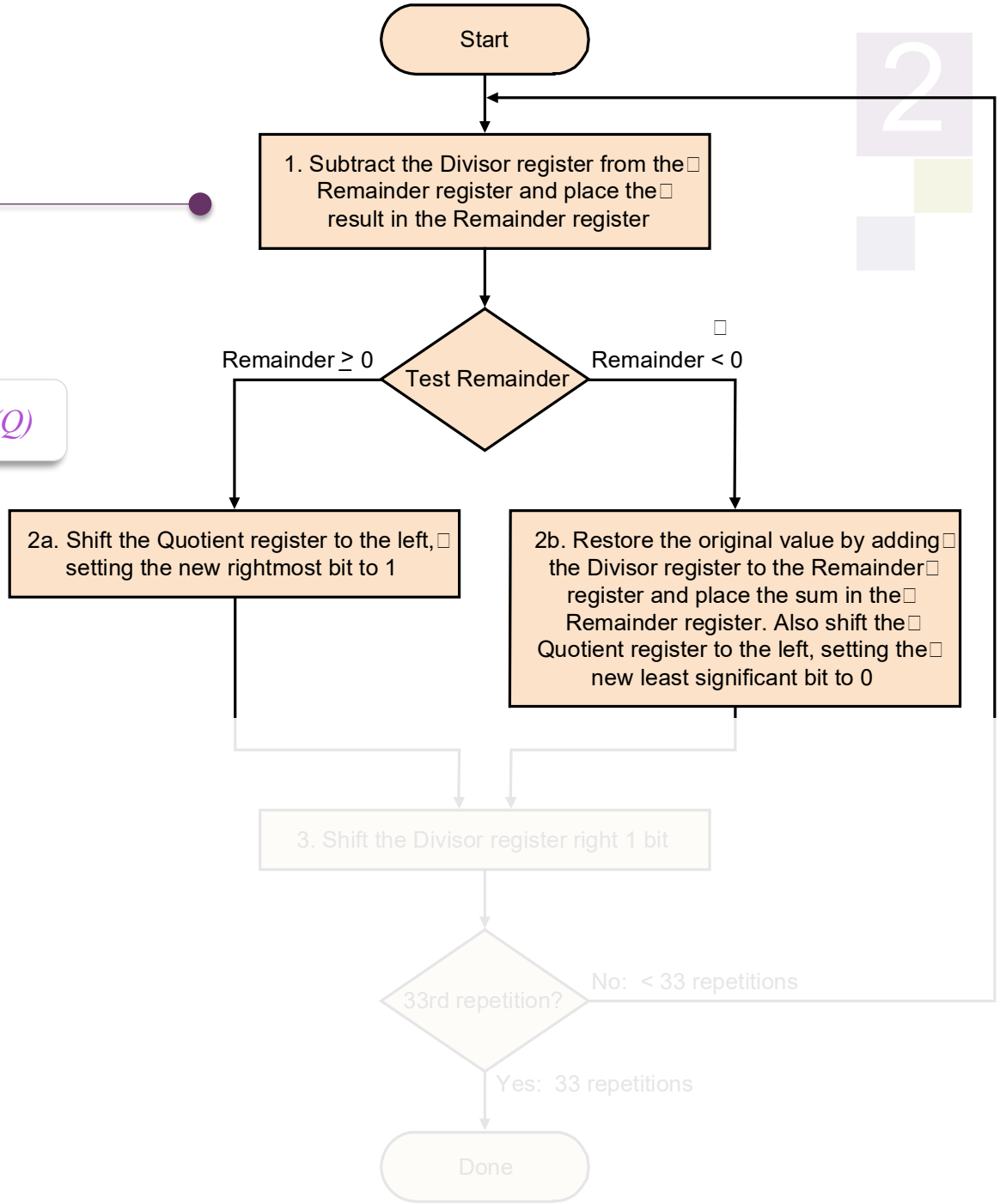
Example 16:

Using 4-bit numbers, divide 7_{10} by 2_{10} .

$7 / 2 = ?$ — *Quotient (Q)*

$0111_2 / 0010_2$

Dividend (DD) — *Divisor (D)*



7 ÷ 2 (0111 ÷ 0010)

Iteration	Step	Quotient (Q)	Divisor (D)	Remainder (R)
		Quotient initialized with zero	Divisor starts at left half of divisor register	Remainder initialized with dividend at right
0	Initial value	0000	0010 0000	0000 0111
1	1. R = R - D			1110 0111
	2b. R < 0; R = D + R			
	Q = Shift left (add 0 at LSB)			
	3. D = Shift right			
2	1. R = R - D			
	2b. R < 0; R = D + R			
	Q = Shift left (add 0 at LSB)			
	3. D = Shift right			

R = R – D = R + (-D)

Negate 0010 0000
➔ 1110 0000

0000 0111
1110 0000 +

1110 0111

7 ÷ 2 (0111 ÷ 0010)

Iteration	Step	Quotient (Q)	Divisor (D)	Remainder (R)
0	Initial value	0000	0010 0000	0000 0111
1	1. R = R - D			1110 0111
	2b. R < 0; R = D + R			0000 0111
	Q = Shift left (add 0 at LSB)	0000		
	3. D = Shift right		0001 0000	
2	1. R = R - D			
	2b. R < 0; R = D + R			
	Q = Shift left (add 0 at LSB)			
	3. D = Shift right			

R = R – D = R + (-D)

Negate 0001 0000
➔ 1111 0000

0010 0000
1110 0111 +

0000 0111

0000 0111
1111 0000 +

1111 0111

7 ÷ 2 (0111 ÷ 0010)

Iteration	Step	Quotient (Q)	Divisor (D)	Remainder (R)
0	Initial value	0000	0010 0000	0000 0111
1	1. R = R - D			1110 0111
	2b. R < 0; R = D + R			0000 0111
	Q = Shift left (add 0 at LSB)	0000		
	3. D = Shift right		0001 0000	
2	1. R = R - D			1111 0111
	2b. R < 0; R = D + R			0000 0111
	Q = Shift left (add 0 at LSB)	0000		
	3. D = Shift right		0000 1000	

0001 0000
1111 0111 +

0000 0111

7 ÷ 2 (0111 ÷ 0010)

Iteration	Step	Quotient (Q)	Divisor (D)	Remainder (R)
0	Initial value	0000	0010 0000	0000 0111
1	1. R = R - D			1110 0111
	2b. R < 0; R = D + R			0000 0111
	Q = Shift left (add 0 at LSB)	0000		
	3. D = Shift right		0001 0000	
2	1. R = R - D			1111 0111
	2b. R < 0; R = D + R			0000 0111
	Q = Shift left (add 0 at LSB)	0000		
	3. D = Shift right		0000 1000	

7 ÷ 2 (0111 ÷ 0010)

Iteration	Step	Quotient (Q)	Divisor (D)	Remainder (R)
		0000	0000 1000	
3	1. R = R – D			1111 1111
	2b. R < 0; R = D + R			0000 0111
	Q = Shift left (add 0 at LSB)	0000		
	3. D = Shift right		0000 0100	
4	1. R = R - D			0000 0011
	2a. R >=0; R = no operation			
	Q = Shift left (add 1 at LSB)	0001		
	3. D = Shift right		0000 0010	
5	1. R = R - D			0000 0001 1
	2a. R >=0; R = no operation			
	Q = Shift left (add 1 at LSB)	0011 3		
	3. D = Shift right		0000 0001	

7 ÷ 2 = 3 remainder 1

Module 2

Data Representation in Computer Systems

2.1 Introduction
2.2 Fixed-Number (Intege
Representation
2.3 Fixed-Number (Intege
Arithmetic
2.4 Floating -Points
Representation
2.5 Floating-Points Arithm
2.6 Summary

- Representation
- Components
- Normalized & Unnormalized
- Format: A Simple Model
- The IEEE-754 Floating-Point Standard

2.4 Floating-Point Representation

- The signed magnitude, one's complement, and two's complement representation that we have just presented deal with integer values only.
- Without modification, these formats are **not useful** in scientific or business applications that deal with real number values.

Real Numbers are just numbers like:

1	12.38	−0.8625	$\frac{3}{4}$	π (pi)	198

In fact:

Nearly any number you can think of is a Real Number

- Floating-point representation solves this problem.

Representation

- Floating-point numbers allow an arbitrary number of decimal places to the right of the decimal point.
  - **Examples**:
  -

$$5 \times 25 = 125$$
$$0.5 \times 0.25 = 0.125$$

→ often expressed in scientific notation as:

$$125 = 1.25 \times 10^{2}$$
$$0.125 = 1.25 \times 10^{-1}$$

Components

Sign

Significand/Fraction/Mantissa

Base/Radix

Exponent

(Decimal) $+743.059 \times 10^{-63}$

(Binary) -101.001×2^{011}

Fraction and *exponent* can be +ve or –ve

Normalized & Unnormalized

- In general mathematics, a floating-point number is said to be *normalized* if the number after the radix point is a non-zero value.
- *Unnormalized* floating number is when the number after the radix point is '0'.
- **Examples**:

$$1.03 \times 10^{-9} \quad (Unnormalized)$$
$$0.10 \times 10^{8} \quad (Normalized)$$
$$45.07 \times 10^{-10} \quad (Unnormalized)$$
$$0.1234 \times 10^{16} \quad (Normalized)$$

Normalization Process

- *Normalization* is the process of deleting the zeroes until a non-zero value is detected.

- **Examples**:

$$0.00743 \times 10^4 = 0.743 \times 10^{4-2}$$
$$= 0.743 \times 10^2$$

$$45.09 \times 10^4 = 0.4509 \times 10^{4+2}$$
$$= 0.4509 \times 10^6$$

A rule of thumb:

- moving the radix point to the right → subtract exponent
- moving the radix point to the left → add exponent

Examples:

- Decimal:

$$743.09 \times 10^{61} = 0.74309 \times 10^{61+3}$$
$$= 0.74309 \times 10^{64}$$

- Binary:

$$10.0111_2 \times 2^{-110011} = 0.100111_2 \times 2^{(-110011)+010}$$
$$= 0.100111_2 \times 2^{-110001}$$

$$0.00011011_2 \times 2^{9} = 0.11011_2 \times 2^{9-3}$$
$$= 0.100111_2 \times 2^{6}$$

Can represent the exponent as decimal.

Format #1 : A Simple Model

- In digital computers, floating-point numbers consist of three parts: a *sign bit*, an *exponent* part (representing the exponent on a power of 2), and a fractional part called a *significand.*

- The general form of *normalized* floating-point is:

$$\pm 0.Fraction \times Base^{\pm exponent}$$

- In binary form:



Format #1: Example

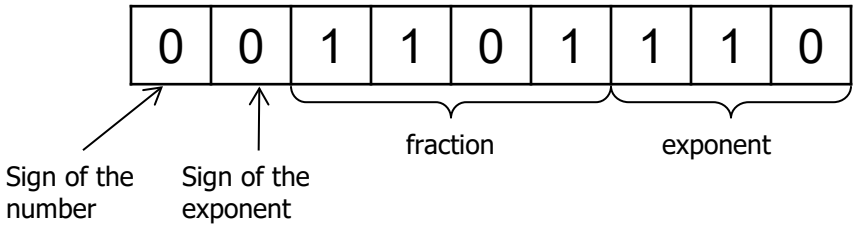
$0.75 \times 2 = 1.5 \rightarrow 1$
$0.5 \times 2 = 1.0 \rightarrow 1$
$0.5 \times 0 = 0.0 \rightarrow 0$

9 bit-hypothetical word

- the first bit is used for the sign of the number,
- the second bit for the sign of the exponent,
- the next four bits for the fraction, and
- the next three bits for the exponent

$$(54.75)_{10} = (110110.11)_2 = (0.11011011)_2 \times 2^6$$
$$= (0.1101)_2 \times (110)_2$$

We have the representation as



2

The 2 signs bit are not good for design as it incurs extra cost.
→ *need new representation.*

- In binary form:





Figure : Example of 14-bit model for floating-point format.

- The next few examples and exercises will be used this simple model before the discussion on the IEEE-754 floating-point standard.

Example 17:

Store the decimal number 17 in this model.

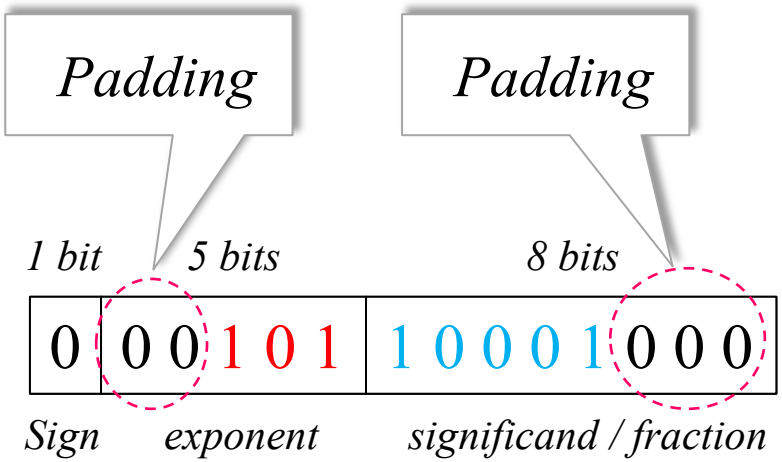
Solution:

(Unnormalized Binary):

$17 = 10001.0 \times 2^0$

(Normalized Binary):

$= 1000.1 \times 2^1$
$= 100.01 \times 2^2$
$= 10.001 \times 2^3$
$= 1.0001 \times 2^4$
$= 0.10001 \times 2^5$

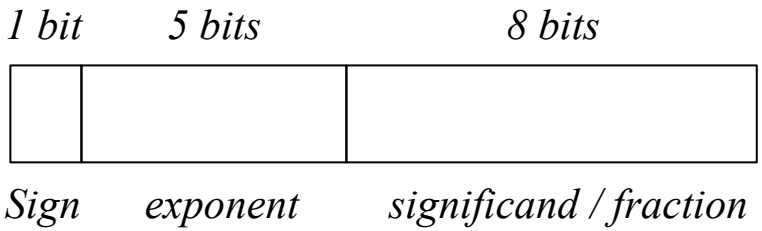


Rule of thumb:

- the *exponent* is always padded to the left (←).
- the *fraction* is always padded to the right (→).

Exercise 2.7:

Based on the following format, write the decimal number 65536 in the floating-point representation and store it in the floating-point format. Show your working.

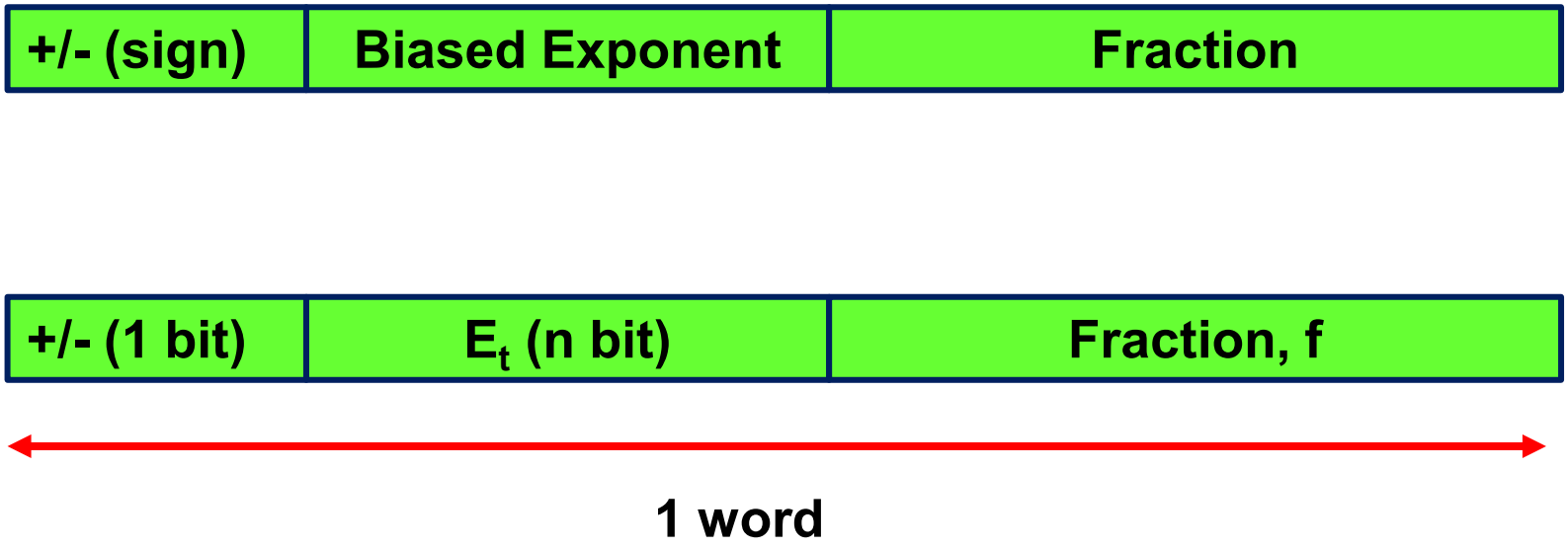


- One obvious problem with previous simple model is that negative exponents OR fractions cannot be represented at the same time.
- **Solution** : *biased* exponent

1 bit	*n bits*	
± *Sign*	*Biased exponent* (E_b)	*significand / fraction*

Format #2: Biased Exponent

- A new format to represent the exponent without the sign bit is introduced
  - This will eliminate the sign for the exponent value that is the exponent will be positive. (indicative)





Biased value, $b = 2^{n-1}$

Biased exponent, $E_t = e' + b$

Normalized exponent, $e' = E_t - b$

Where,
E_t = biased exponent
n = no. of bits of exponent field

This is used unless the IEEE standard is mentioned – then is a different calculation

- The steps of conversion to floating-point:
  - Change to binary (if given decimal number).
  - Normalized the number.
  - Change the number to *biased exponent*.
  - Form the floating-point format (3 fields).

Example of conversion steps

- Transform -33.625 to floating point word using the following format (radix 2)



This is the given word format

- Step 1 : Change 33.625 to binary

33 ➔ 0100001

0.625 x 2 = 1.25 ➔ 1
0.25 x 2 = 0.5 ➔ 0
0.5 x 2 = 1.0 ➔ 1
0.0

0.625 = .101

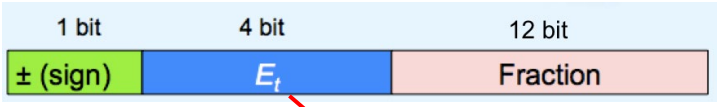
33.625
= **0100001.101**

- Step 2 : Normalized the number

Normalized exponent, e'

0100001.101_2 → $0.100001101_2 \times 2^{0110}$

- Step 3: Change the number to biased exponent



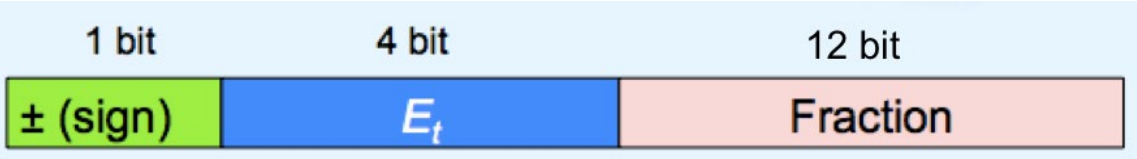
Biased value, $b = 2^{n-1} = 2^{4-1} = 8_{10} = 1000_2$

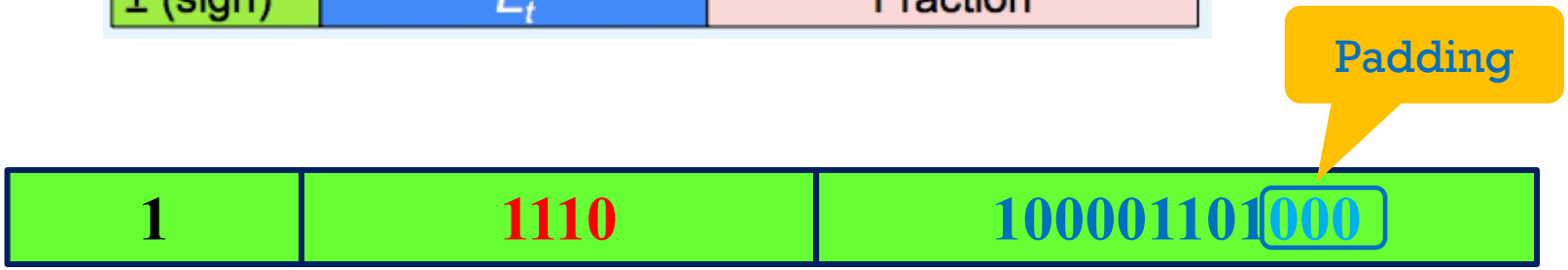
Biased exponent, $E_t = e' + b = 0110 + 1000 = 1110$

$0.100001101_2 \times 2^{0110}$ → $0.100001101_2 \times 2^{1110}$

$0.100001101 \times 2^{1110}$ -33.625

- Step 4 : Form the word (3 fields)

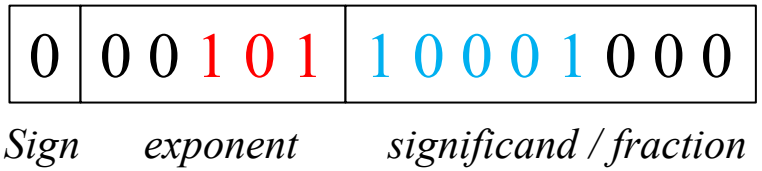




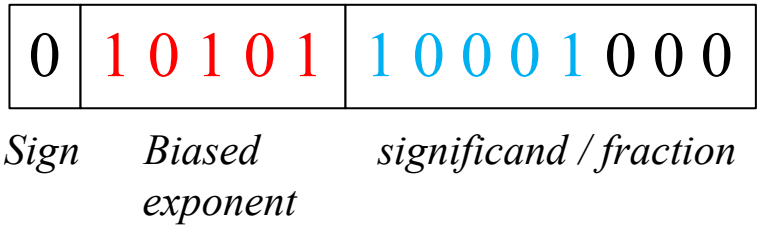
Rule of thumb:
- the biased exponent (E_t) is always padded to the left
- the fraction is always padded to the right

Example 18:

Returning to *Example 17* that storing: $\mathbf{17} = 0.10001 \times 2^5$



- The *bias* value, $b = 2^{5-1} = 2^4 = 16$
- The *biased exponent*, E_b is now = $e' + b = 5 + 16 = 21$



Example 19:

Store the decimal number 0.25 in the floating-point representation.

Show your working.

Solution:

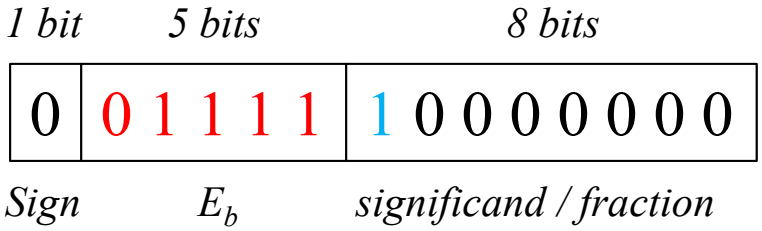
(Convert into binary using repetitive multiplication):

$$0.25 \times 2 = 0.5$$
$$0.5 \times 2 = 1.0$$
$$0.25_{10} = 0.01_2$$

(Normalized Binary):

$$0.25_{10} = 0.01 \times 2^0$$
$$= 0.1 \times 2^{-1}$$

- The *bias* value, $b = 2^{5-1} = 16$
- The *biased exponent*, E_b is now 16 + (–1) = 15



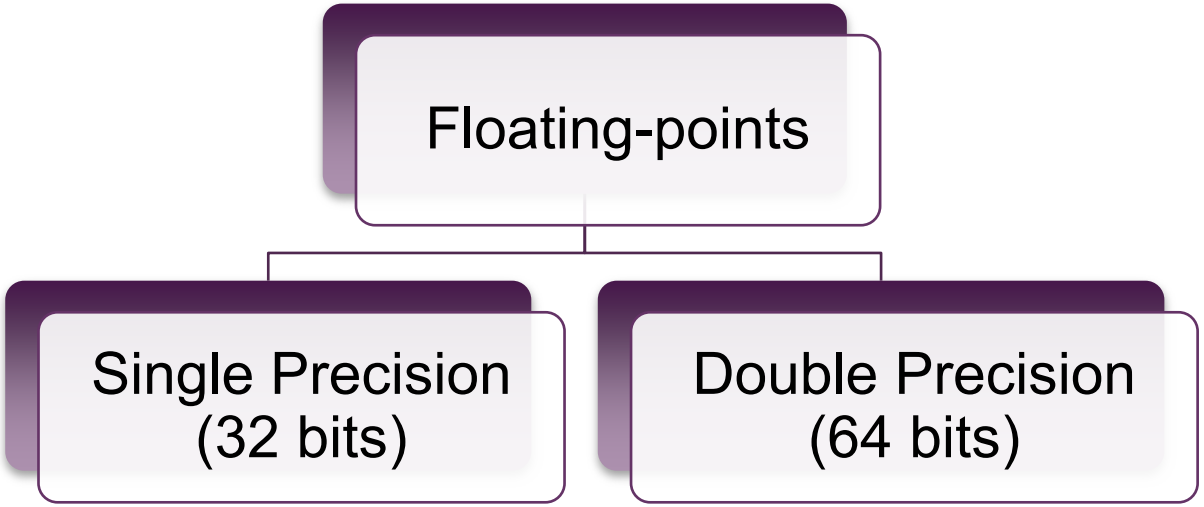
Format #3: IEEE Floating Point standard



The IEEE-754 Floating -point Standard

(IEEE) Institute of Electrical and Electronic Engineers

- The floating-point model (non-standard) described before is for simplicity and conceptual understanding.
- The official standard is known as IEEE-754 (1985):



The IEEE-754 Floating-point Standard

Representation

- The representation of normalized floating-point in IEEE standard:

$$(-1)^{Sign} \times (1 + Fraction) \times 2^{Exponent}$$

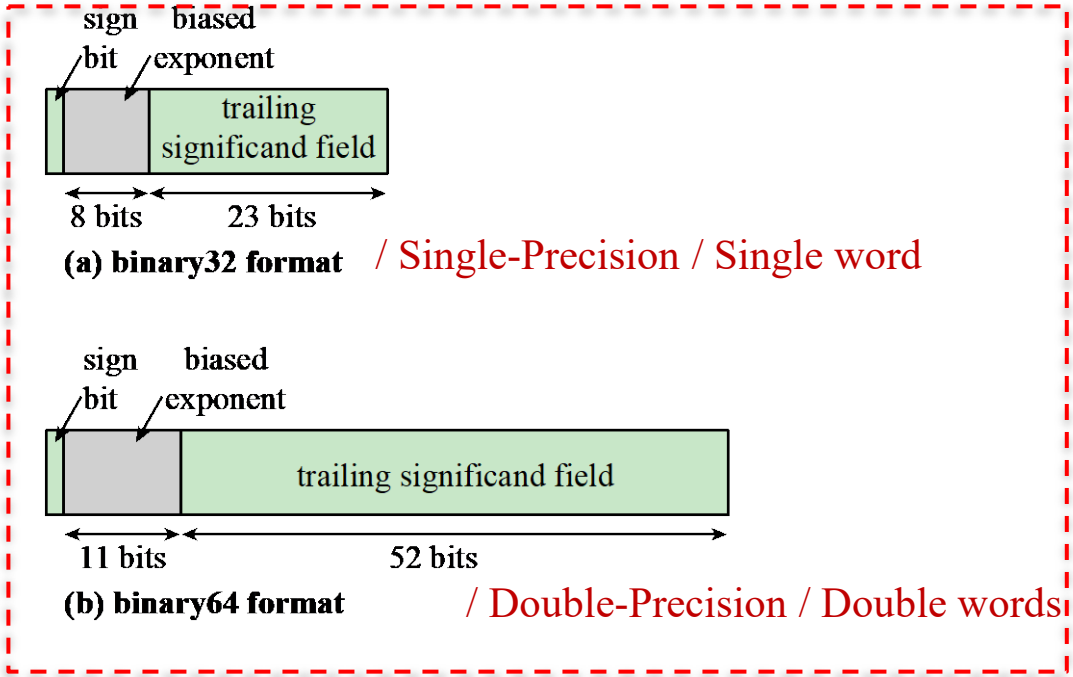
Sign (S):

+ve (0) or

-ve (1)

The IEEE-754 Floating-point Standard

The word format ...



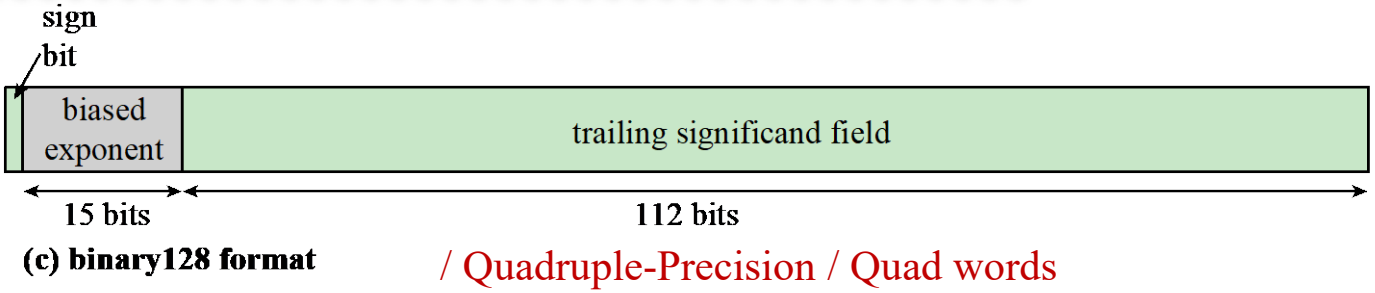


Table 10.3 IEEE 754 Format Parameters

Parameter	Format		
	binary32	**binary64**	**binary128**
Storage width (bits)	32	64	128
Exponent width (bits)	8	11	15
Exponent bias	127	1023	16383
Maximum exponent	127	1023	16383
Minimum exponent	–126	–1022	–16382
Approx normal number range (base 10)	10_{-38}, 10_{+38}	10_{-308}, 10_{+308}	10_{-4932}, 10_{+4932}
Trailing significand width (bits)*	23	52	112
Number of exponents	254	2046	32766
Number of fractions	2_{23}	2_{52}	2_{112}
Number of values	$1.98 \times 2_{31}$	$1.99 \times 2_{63}$	$1.99 \times 2_{128}$
Smallest positive normal number	2_{-126}	2_{-1022}	2_{-16362}
Largest positive normal number	$2_{128} - 2_{104}$	$2_{1024} - 2_{971}$	$2_{16384} - 2_{16271}$
Smallest subnormal magnitude	2_{-149}	2_{-1074}	2_{-16494}

* not including implied bit and not including sign bit

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

The IEEE-754 Floating-point Standard

Normalized & Unnormalized

- In IEEE standard normalization (used in computers), a floating point number is said to be *normalized* if there is only a single **non-zero** before the radix point.

- **Examples**:

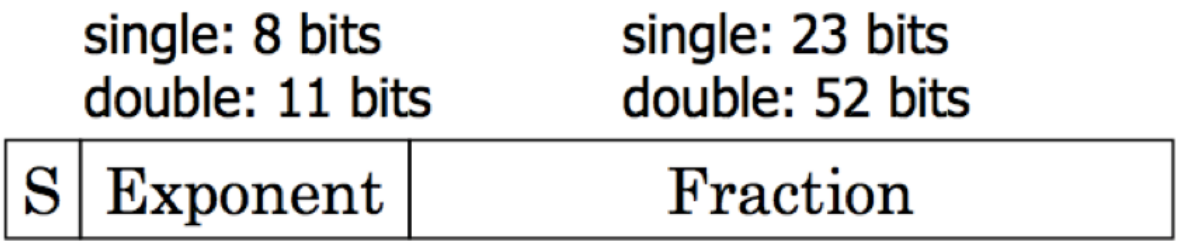
$1.03_{10} \times 10^{-9}$ (Normalized)

$0.10_{10} \times 10^{8}$ (Unnormalized)

$1.011_{2} \times 2^{-011}$ (Normalized)

$1.234_{10} \times 10^{16}$ (Normalized)

Biased Notation IEEE-754



Exponent:

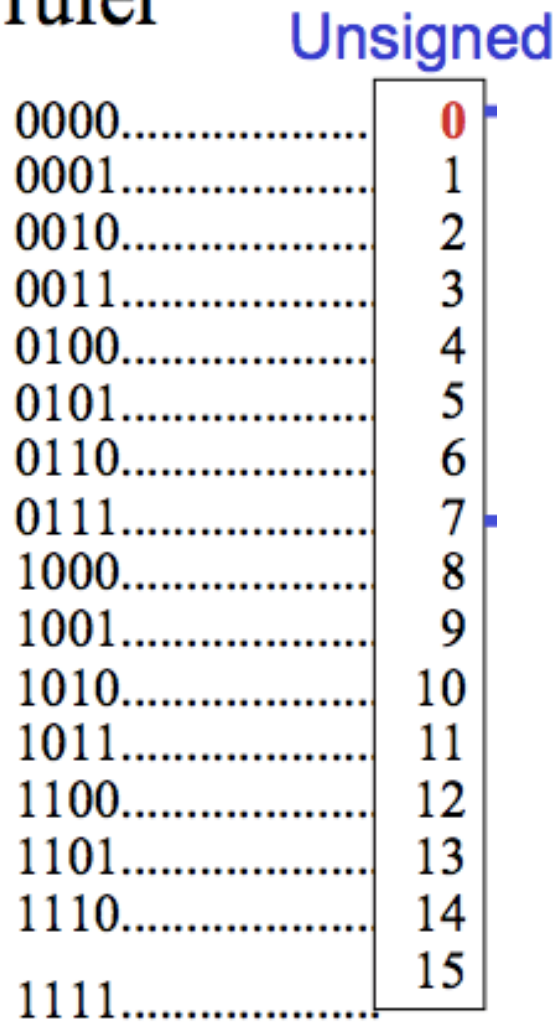
- use excess-k representation (as a biased exponent)
- Single precison, E = 8-bit, Bias = 127
- Double precision, E =11-bit, Bias = 1023

Excess-*k* representation

- Yet another way to represent *signed numbers* in binary.
- For N bit numbers, k is $2^{N-1}-1$
- So for 4-bit integers, k is 7
  - i.e. $2^{4-1}-1 = 2^3 - 1 = 8 - 1 = 7$
- The value of each signed binary number is
  - *'unsigned value'* – k

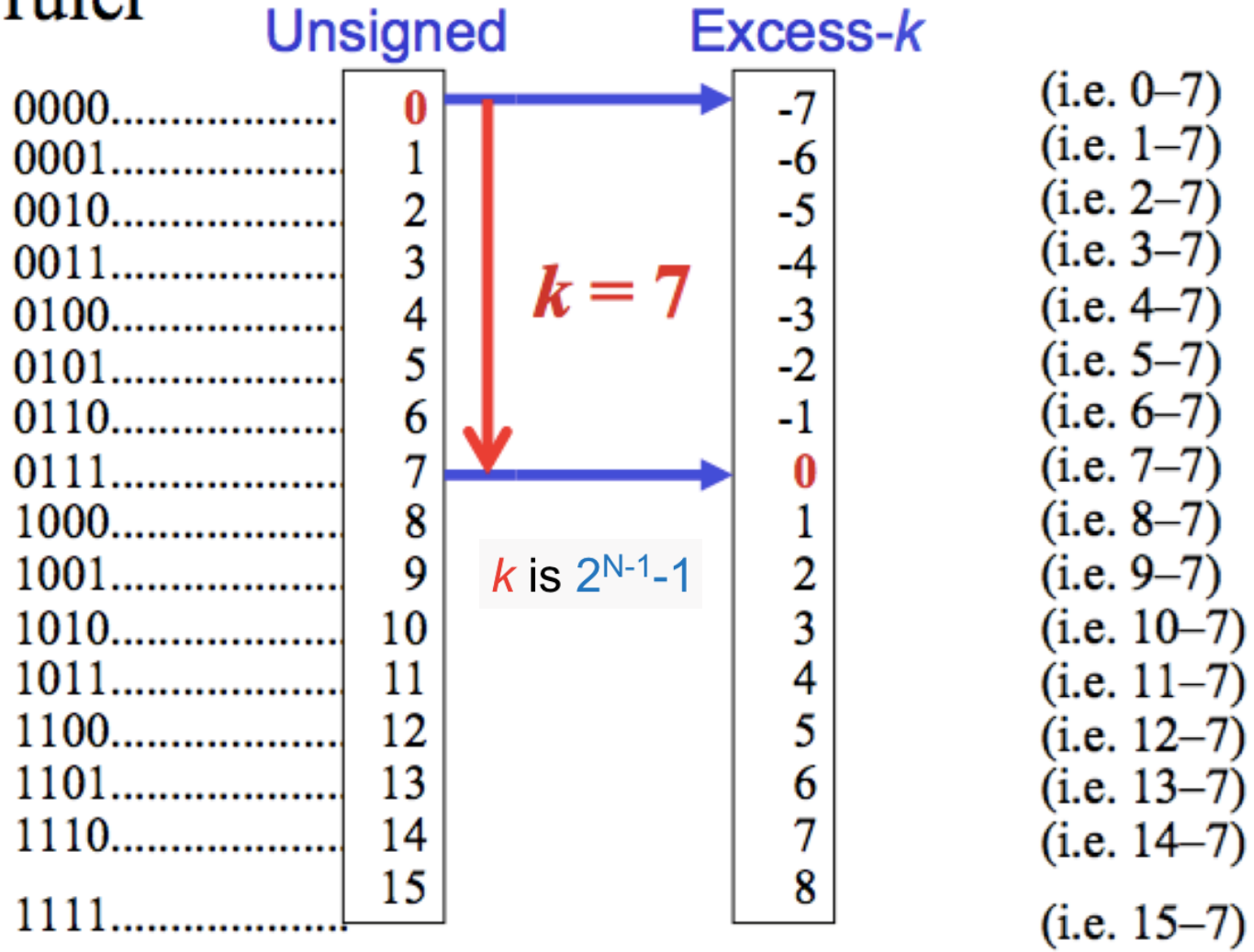
Excess-*k* representation: 4-bit (i.e. excess 7)

"Sliding ruler"



Signed ???

"Sliding ruler"



Excess-*k* representation: Example

- Convert **-14** (decimal) to 8-bit excess-*k*
- What is ***k*** ?
  k = $2^{N-1}-1 = 2^{8-1}-1 =$ **127** (i.e. excess-127)
- Find the number ***u*** such that ***u*** - ***k*** = -14.
  - ***u*** - **127** = -14,
  - ***u*** = -14 + 127 = **113**
- Convert ***u*** to binary:
  01110001

8-bit ; excess-127

8 bit excess-127

Binary value	Excess-127 interpretation	Unsigned interpretation
00000000	−127	0
00000001	−126	1
⋮	⋮	⋮
01111111	0	127
10000000	1	128
10000001	2	129
⋮	⋮	⋮
11111111	+128	255

8-bit ; excess-127

e		Meaning
0000 0000		Reserved
0000 0001		-126_{10}
0000 0010		-125_{10}
0111 1111		0_{10}
1111 1110		127_{10}
1111 1111		Reserved

The IEEE-754 Floating-point Standard

Conversion of Decimal to Binary Floating-point

- To convert a decimal number to single or double precision floating point:
  - Step 1: Normalize.
  - Step 2: Determine sign bit.
  - Step 3: Determine exponent.
  - Step 4: Determine significand (fraction)

IEEE 754 Conversion : Example 20

2

- Convert 10.4_d to *single precision* floating point.
- **Step 1: Normalize**

10 → 1010

0.4 x 2 = 0.8 → 0
0.8 x 2 = 1.6 → 1
0.6 x 2 = 1.2 → 1
0.2 x 2 = 0.4 → 0
0.4 x 2 = 0.8 → 0
0.8 x 2 = 1.6 → 1

For continuous results, take the 1st pattern before it repeats itself

0.4 = .0110

10.4 = 1010.0110 x 2^0

→ **1.0100110 x 2^3**

IEEE 754 Conversion : Example

2

- **Step 2: Determine Sign Bit (S)**
  - Because (10.4) is positive, S = **0**

- **Step 3: Determine exponent**
  - Because its single precision ➔ bias = 127
  - Exponent= 3 + bias

$= 3 + 127$

$= 130_d$

$= \mathbf{1000\ 0010_b}$

➔ **1.0100110 x 2^3**

➔ **1.0100110 x 2^{130}**

Biased exponent,
$E_t = e' + b$

IEEE 754 Conversion : Example

- **Step4: Determine fraction**
  - Drop the leading 1 of the significand

1.0100110×2^{130} ➔ **0100110**

  - Then expand (padding) to 23 bits

01001100000000000000000

  - Complete the representation:

sign	Exponent	Fraction
0	10000010	01001100000000000000000

Exponent = $1000\ 0010_b$

2

The IEEE-754 Floating -point Standard

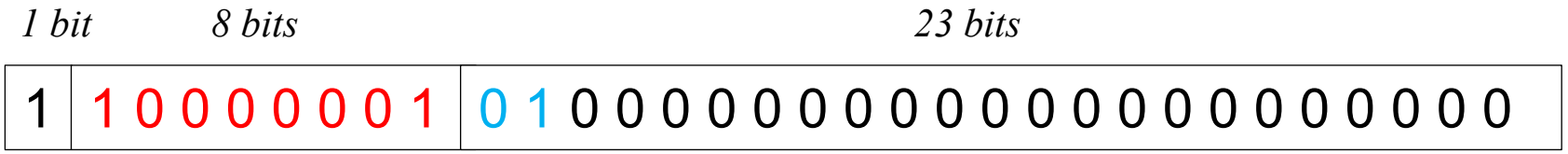
Conversion of Binary Floating-point to Decimal

- To convert a single or double precision floating point to decimal number:
  - **Step 1**: Extract value of sign.
  - **Step 2:** Extract value of *biased exponent* & bias value.
  - **Step 3**: Extract value of fraction.
  - **Step 4**: Apply the basic equation.

2

Example 21:

What is the decimal number represented by this single precision float?



Solution:

- **Step 1**: Extract value of sign. → $S = 1$
- **Step 2**: Extract value of biased exponent, E_B → 10000001 = 129_{10}

  Bias value, B → $(2^{n-1} - 1) = 2^7 - 1 = 128 - 1 = 127_{10}$

0 0 1	0 1 0 0 0

→ 01 x 2^0
→ 0.1 x 2^{-1}
→ 1.0 x 2^{-2}

$S = 1$
$Fraction = 0.25$
$E_B = 129$
$B = 127$

- **Step 3**: Extract value of fraction. → 1 x 2^{-2} $= \frac{1}{2^2} = \frac{1}{4} = 0.25_{10}$

- **Step 4**: Apply the basic equation.

$$(-1)^S \times (1 + Fraction) \times 2^{e'}$$

$E_B = e' + \text{bias}$
→ $e' = E_B - B$

= (– 1)^1 x (1 + 0.25) x $2^{129 - 127}$
= (– 1) x (1.25) x 2^2
= (– 1.25) x 4
= – 5.0_{10}

Module 2

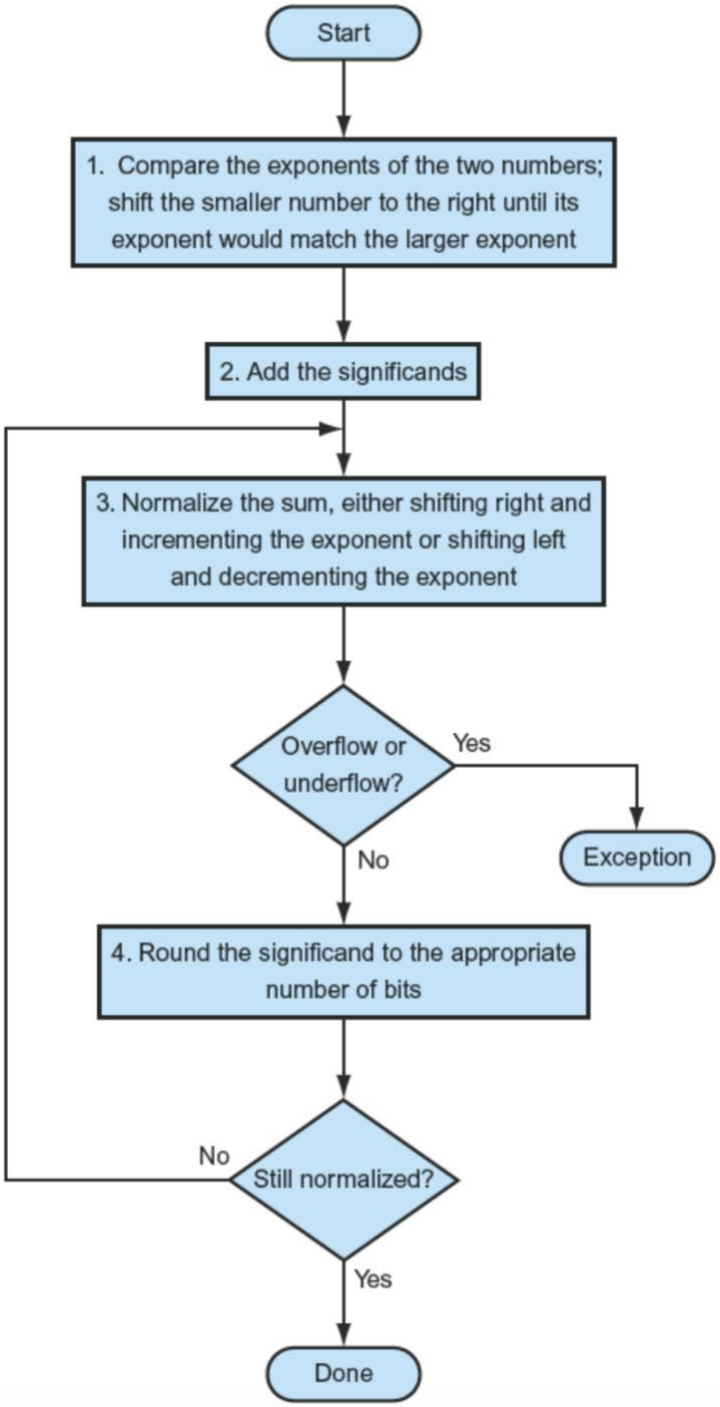
Data Representation in Computer Systems

2.1 Introduction
2.2 Fixed-Number (Integer) Representation
2.3 Fixed-Number (Integer) Arithmetic
2.4 Floating-Points Representation
2.5 Floating -Points Arithmetic
2.6 Summary

- Overview
- Addition
- Multiplication

2 Addition

Flows



Biased Exponent ranges:

Single precision: -126 ~ 127

Double precision: - 1022 ~ 1023

Patterson, D.A. and Hennessy, J.L. (2014). *Computer Organization and Design: The Hardware/Software Interface (5th Edition)*. United States: Elsevier, p.205.

- Steps for addition from the previous flow chart:
  - **Step 1**: Align the decimal point of the number that has the smaller *exponent*.
  - **Step 2**: Add the fraction.
  - **Step 3**: Normalize the sum.
  - **Step 4**: Round the fraction.
    (If the fraction does not fit in the space reserved for it, it has to be rounded off)
  - **Step 5**: Normalize it (if need be) .

Example 22:

Add these two decimal floating-point numbers. Assume that we can store only four decimal digits of the significand and two decimal digits of the exponent.

$(9.999_{10} \times 10^{1}) + (1.610_{10} \times 10^{-1}) = _____{}_{10}$

Solution:

- **Step 1**: Align the decimal point of the number that has the smaller exponent.

$1.610_{10} \times 10^{-1}$

$= 1.610_{10} \times 10^{-1} \times 10^{2}$

$= 0.0161_{10} \times 10^{1}$

- **Step 2**: Add the fraction.

$$\begin{array}{r} 9.9990 \times 10^{1} \\ +\ 0.0161 \times 10^{1} \\ \hline 10.0151 \times 10^{1} \end{array}$$

- **Step 3**: Normalize the sum.

$$10.0151 \times 10^1 \rightarrow 1.00151 \times 10^2$$

- **Step 4:** Round the fraction (to 4 decimal digits for fraction).

$$1.00151 \times 10^2 \rightarrow 1.0015 \times 10^2$$

- **Step 5**: Normalize it (if need be).

No need as its normalized

Answer = 1.0015×10^2

Example 23:

Add these two decimal numbers in binary floating-point form:

$$0.5_{10} + (-\ 0.4375_{10}) = _____\ _2$$

Solution: (Convert to binary)

$0.5 \times 2 = 1.0$

$(0.1_2 \times 2^0)$

$0.4375 \times 2 = 0.875$
$0.875 \times 2 = 1.75$
$0.75 \times 2 = 1.5$
$0.5 \times 2 = 1.0$

$(-\ 0.0111_2 \times 2^0)$

$$(1.0_2 \times 2^{-1}) + (-\ 1.11_2 \times 2^{-2})$$

- **Step 1**: Align the decimal point of the number that has the smaller exponent.

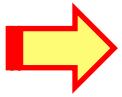
  $-1.11_2 \times 2^{-2}$

  $= -1.11_2 \times 2^{-2} \times 2^{1}$

  $= -0.111_2 \times 2^{-1}$

- **Step 2**: Add the fraction.

$$\begin{array}{rl} & 1.000 \times 2^{-1} \\ + & -0.111 \times 2^{-1} \\ \hline & \\ \hline \end{array}$$



$$\begin{array}{rl} & \cancel{1.000} \times 2^{-1} \\ - & 0.111 \times 2^{-1} \\ \hline & 0.001 \times 2^{-1} \end{array}$$

- **Step 3**: Normalize the sum.

  $0.001 \times 2^{-1} \times 2^{-3} \rightarrow 1.0 \times 2^{-4}$

- **Step 4:** Round the fraction (to 4 decimal digits for fraction).

  $1.0 \times 2^{-4} \rightarrow 1.0000 \times 2^{-4}$

- **Step 5**: Normalize it (if need be).

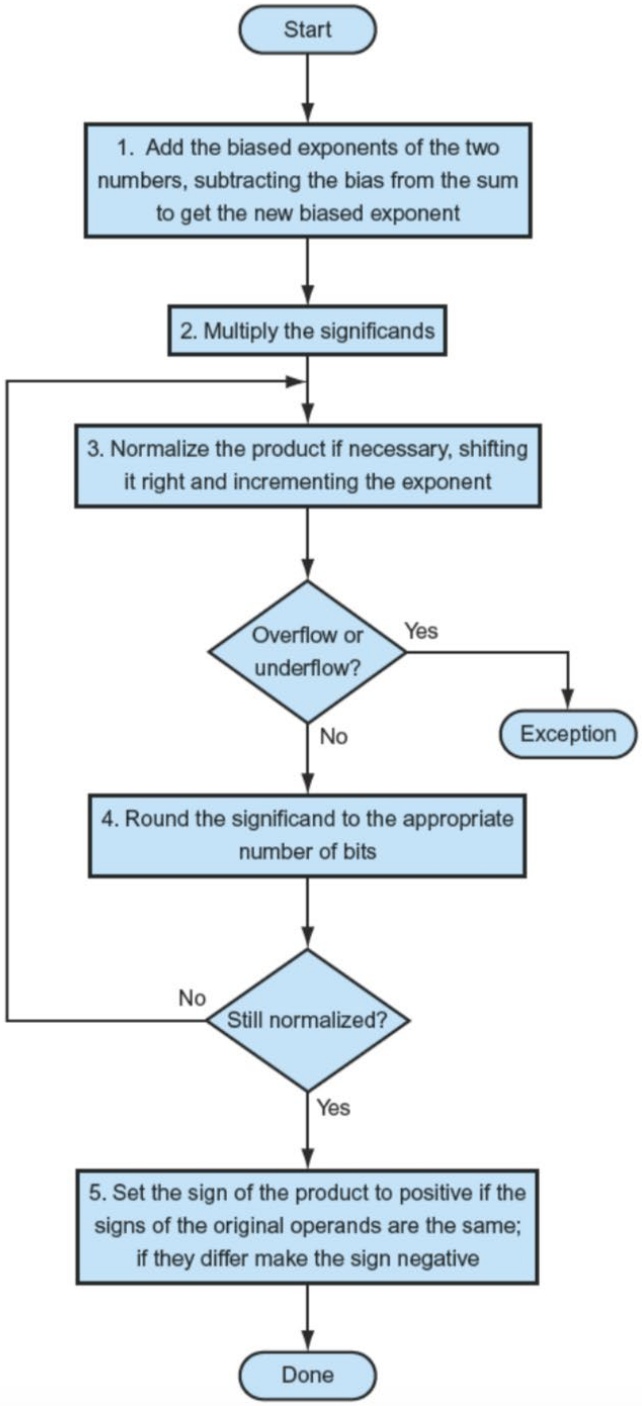
No need as its normalized

Answer = 1.0000×2^{-4}

$$.(0 \times 2^{-1}) + (0 \times 2^{-2}) + (0 \times 2^{-3}) + (1 \times 2^{-4})$$
$$.(0.\frac{1}{2^1}) + (0.\frac{1}{2^2}) + (0.\frac{1}{2^3}) + (1.\frac{1}{2^4})$$
$$.(0.\frac{1}{2}) + (0.\frac{1}{4}) + (0.\frac{1}{8}) + (1.\frac{1}{16})$$

The result in decimal is then:

$$1.0000 \times 2^{-4} = 0.0001000_2 = 0.0001_2$$
$$= \frac{1}{(2^4)_{10}} = \frac{1}{16_{10}} = 0.0625_{10}$$



2 Multiplication

Flows

Patterson, D.A. and Hennessy, J.L. (2014). *Computer Organization and Design: The Hardware/Software Interface (5th Edition)*. United States: Elsevier, p.209.

- Steps for multiplication from the previous flow chart:
  - **Step 1**: Add the *exponent* of the 2 numbers.
  - **Step 2**: Multiply the fraction.
  - **Step 3**: Normalize the product.
  - **Step 4**: Round the fraction.

    (If the fraction does not fit in the space reserved for it, it has to be rounded off)
  - **Step 5**: Normalize it (if need be) .
  - **Step 6**: Set the sign of the product.

Example 24:

2

Multiply these two decimal floating-point numbers.

Assume 4 decimal digits for *significand* and 2 decimal digits for *exponent*.

$$(1.110 \times 10^{10}) \times (9.200 \times 10^{-5}) = ______{10}$$

Solution:

- **Step 1**: Add the exponent of the 2 numbers.

  10 + (–5) = 5

  If bias is considered →
  5 + 127 = 132

- **Step 2**: Multiply the fraction.

```
    9.200
x   1.110
    0 000
   92 00
  920 0
 9200
10212000
```

→ 10.212000
= 10.2120 x 10^5

2

- **Step 3**: Normalize the product.

$$10.2120 \times 10^5 \times 10^1 \rightarrow 1.02120 \times 10^6$$

- **Step 4:** Round the fraction (to 4 decimal digits for fraction).

$$1.02120 \times 10^6 \rightarrow 1.0212 \times 10^6$$

- **Step 5**: Normalize it (if need be).

No need as its normalized

- **Step 6**: Set the sign of the product.

$$+1.0212 \times 10^6$$

Example 25:

Multiply these two binary floating-point numbers.

Assume 4 binary digits for significand and 2 binary digits for exponent.

$$(1.000 \times 2^{-1}) \times (-1.110 \times 2^{-2}) \quad = _____ _2$$

Solution:

- **Step 1**: Add the exponent of the 2 numbers.

  (–1) + (–2) = –3

If bias is considered →

(–3) + 127 = 124

- **Step 2**: Multiply the fraction.

```
    1.110
x   1.000
    0 000
   00 00
  000 0
 1110
1110000
```

→ 1.110000
= 1.110000 x 2^{-3}

- **Step 3**: Normalize the product.

Already normalized

- **Step 4:** Round the fraction (to 4 decimal digits for fraction).

$1.110000 \times 2^{-3} \rightarrow 1.1100 \times 2^{-3}$

- **Step 5**: Normalize it (if need be).

No need as its normalized

- **Step 6**: Set the sign of the product.

-1.1100×2^{-3}

$$-111_2 \times 2^{-5} = -7_{10} \times \frac{1}{2^5}$$
$$= -(\frac{7}{32}) = -0.21975_{10}$$

2.6 Summary

2

- This module presented the essentials of data representation and numerical operations in digital computers.
- Student should master the techniques described for base conversion and memorize the smaller hexadecimal and binary numbers.
- This knowledge will be beneficial to student throughout remainder of this subject.
- Knowledge of hexadecimal coding will be useful if you are ever required to read a core (memory) dump after a system crash or if do any serious work in the field of data communications.

Floating-points

Non-Standard

(*Example*: 14-bits)

Standard (IEEE-754)

(32-bits) Single Precision

(64-bits) Double Precision

Normalized:

$0.12_{10} \times 10^{3}$

$0.1111_{2} \times 2^{7}$

1	5	8
S	*e'*	*Fraction*

$\pm 0.Fraction \times Base^{e'}$

$1.2_{10} \times 10^{2}$

$1.111_{2} \times 2^{6}$

1	8	23
S	*e'*	*Fraction*

1	11	52
S	*e'*	*Fraction*

$(-1)^{S} \times (1 + Fraction) \times Base^{e'}$

Biased Notation:

$\pm 0.Fraction \times Base^{E_b}$

$b = 2^{n-1}$

$b = 16$ $\quad E_b = e' + b$

$(-1)^{S} \times (1 + Fraction) \times Base^{E_B}$

$B = (2^{n-1}) - 1$

$B = 127$ $\quad E_B = e' + B$ $\quad B = 1023$